

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI  
DEPARTMENT OF SPACE AND EARTH SCIENCES



## BACHELOR THESIS

By  
**Tran Anh Duy**

Project

---

### DEVELOPMENT AND IMPLEMENTATION OF AUTONOMOUS FLIGHT CONTROL FOR A VERTICAL TAKE-OFF AND LANDING QUADPLANE UAV

---

**Supervisors:** Assoc. Prof Pham Van Bach Ngoc <sup>1</sup>  
Dr. Nguyen Duc Hung <sup>2</sup>

<sup>1</sup> *Vietnam National Space Center, Vietnam*

<sup>2</sup> *University of Science and Technology of Hanoi, Vietnam*

Hanoi, 28 August 2026

# Table of Contents

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| <b>ACKNOWLEDGEMENTS</b>                                                                   | <b>iii</b> |
| <b>LIST OF ABBREVIATIONS</b>                                                              | <b>iv</b>  |
| <b>Abstract</b>                                                                           | <b>1</b>   |
| <b>1 Introduction</b>                                                                     | <b>2</b>   |
| 1.1 Motivation . . . . .                                                                  | 2          |
| 1.2 Research problem . . . . .                                                            | 2          |
| 1.3 Scope and thesis organisation . . . . .                                               | 5          |
| <b>2 Objectives</b>                                                                       | <b>5</b>   |
| <b>3 Literature Review</b>                                                                | <b>6</b>   |
| 3.1 Fundamentals of aircraft and multirotor attitude control . . . . .                    | 6          |
| 3.2 Reference frames and the six-degree-of-freedom rigid-body model . .                   | 6          |
| 3.2.1 Reference frames . . . . .                                                          | 6          |
| 3.2.2 Twelve states, six degrees of freedom . . . . .                                     | 7          |
| 3.2.3 Rigid-body dynamics . . . . .                                                       | 7          |
| 3.2.4 How many variables control the body frame? . . . . .                                | 8          |
| 3.2.5 Scope of custom controller . . . . .                                                | 8          |
| 3.3 Attitude control architectures across vehicle classes . . . . .                       | 10         |
| 3.3.1 Multirotor rate/attitude control . . . . .                                          | 10         |
| 3.3.2 Fixed-wing attitude and energy control . . . . .                                    | 10         |
| 3.3.3 The quadplane hybrid case and the transition corridor . . .                         | 10         |
| 3.4 Nonlinear dynamic inversion: general formulation and its central<br>problem . . . . . | 11         |
| 3.5 Incremental nonlinear dynamic inversion . . . . .                                     | 12         |
| 3.5.1 Origins and the Taylor-series derivation . . . . .                                  | 12         |
| 3.5.2 Why INDI over PID, gain-scheduled NDI, or MPC . . . . .                             | 14         |
| 3.6 Estimating angular acceleration and the synchronization problem .                     | 15         |
| <b>4 Methodology</b>                                                                      | <b>16</b>  |
| 4.1 Aircraft, autopilot and simulator . . . . .                                           | 17         |
| 4.2 Aerodynamic model: from OpenVSP to Gazebo . . . . .                                   | 20         |

|          |                                                                                            |           |
|----------|--------------------------------------------------------------------------------------------|-----------|
| 4.3      | Rate controllers . . . . .                                                                 | 22        |
| 4.4      | Which effector runs INDI, and when . . . . .                                               | 23        |
| 4.5      | Simulation set-up and flight profiles . . . . .                                            | 24        |
| 4.6      | Wind and disturbance injection . . . . .                                                   | 25        |
| 4.7      | Test configuration and scenarios . . . . .                                                 | 26        |
| 4.8      | Metrics and analysis procedure . . . . .                                                   | 27        |
| <b>5</b> | <b>Results and Discussion</b>                                                              | <b>27</b> |
| 5.1      | What the real flight log can and cannot establish . . . . .                                | 27        |
| 5.2      | Baseline simulated mission path . . . . .                                                  | 30        |
| 5.3      | Aerodynamic model . . . . .                                                                | 30        |
| 5.4      | Identifying the aircraft that is being controlled . . . . .                                | 33        |
| 5.5      | The dominant sim-to-real risk: vibration in the angular-acceleration<br>estimate . . . . . | 35        |
| 5.6      | Verifying that the simulated wind is a wind . . . . .                                      | 37        |
| 5.7      | Yaw authority . . . . .                                                                    | 38        |
| 5.8      | Fixed-wing INDI during transition and forward flight . . . . .                             | 38        |
| 5.9      | Stability and control authority . . . . .                                                  | 41        |
| 5.10     | PID and INDI comparison . . . . .                                                          | 43        |
| 5.11     | Discussion of findings . . . . .                                                           | 46        |
| 5.12     | Limitations of the study . . . . .                                                         | 47        |
|          | 5.12.1 Problems encountered during implementation . . . . .                                | 47        |
| <b>6</b> | <b>Conclusion and Future Work</b>                                                          | <b>49</b> |
| 6.1      | Future work . . . . .                                                                      | 49        |
| <b>A</b> | <b>Physical Prototype Build</b>                                                            | <b>54</b> |
| <b>B</b> | <b>Vehicle and Controller Parameters</b>                                                   | <b>55</b> |
| <b>C</b> | <b>Names Used in ArduPilot, Gazebo and the Project Files</b>                               | <b>56</b> |
| <b>D</b> | <b>Transition Control Hooks</b>                                                            | <b>57</b> |
| <b>E</b> | <b>Gazebo, SITL, and MAVLink Configuration</b>                                             | <b>58</b> |
| <b>F</b> | <b>Additional Derivations and Pseudocode</b>                                               | <b>59</b> |

# ACKNOWLEDGEMENTS

I would like to first express my deepest gratitude to my mentor and supervisor, Assoc. Prof Pham Van Bach Ngoc, for his invaluable guidance, support, and encouragement throughout this project. He has been a source of inspiration and for giving me the chance to learn and grow academically in his research group. This opportunity has helped me meet smart and talented people of diverse backgrounds, fostering both professional growth.

# LIST OF ABBREVIATIONS

The abbreviations below are short for the following full terms.

| Abbreviation | Short for                               |
|--------------|-----------------------------------------|
| UAV          | Unmanned Aerial Vehicle                 |
| VTOL         | Vertical Take-Off and Landing           |
| INDI         | Incremental Nonlinear Dynamic Inversion |
| NDI          | Nonlinear Dynamic Inversion             |
| PID          | Proportional–Integral–Derivative        |
| SITL         | Software-In-The-Loop                    |
| TECS         | Total Energy Control System             |
| EKF          | Extended Kalman Filter                  |
| IMU          | Inertial Measurement Unit               |
| GPS          | Global Positioning System               |
| MAVLink      | Micro Air Vehicle Link                  |
| CAD          | Computer-Aided Design                   |
| DOF          | Degrees of Freedom                      |
| ENU          | East-North-Up                           |
| FRD          | Forward-Right-Down                      |
| NED          | North-East-Down                         |
| MPC          | Model Predictive Control                |
| SDF          | Simulation Description Format           |
| SISO         | Single-Input/Single-Output              |
| OpenVSP      | Open Vehicle Sketch Pad                 |
| UIUC         | University of Illinois Urbana-Champaign |
| RPM          | Revolutions Per Minute                  |
| RMSE         | Root Mean Square Error                  |

# LIST OF TABLES

|    |                                                                                                                                                                                                                                                                                                  |    |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2  | Parameters of one Gazebo lift-drag panel, with the main-wing values of the flown model and the matching OpenVSP value where one exists. The SDF tag names are given in Appendix C. . . . .                                                                                                       | 21 |
| 3  | Flight profiles. Both are read directly from their mission files, so the desired path in every figure and the flown mission come from the same source. . . . .                                                                                                                                   | 25 |
| 4  | Test matrix. Every case uses the same aircraft model and mission; PID and INDI differ only in the INDI parameter block. Wind speed is also given as a fraction of the 17 m/s cruise airspeed. . . . .                                                                                            | 26 |
| 5  | Moment-pulse schedule, conditions 2 and 3. Each window starts a fixed time after an autopilot event, measured on the vehicle clock, so PID and INDI receive the pulses at the same point of the mission. Each window is a roll pulse, 4s rest, then a pitch pulse: eight pulses per run. . . . . | 26 |
| 6  | Lifting surfaces of the flown Gazebo model, ranked by maximum lift. Share is of the wing and horizontal-tail total, 0.428 m <sup>2</sup> . Left and right panels are combined. . . . .                                                                                                           | 32 |
| 7  | Control effectiveness and effector lag, identified in hover from a swept-stick manoeuvre with the rate loop on the PID. . . . .                                                                                                                                                                  | 34 |
| 8  | Angular-acceleration noise in the estimator band, 25 Hz to 90 Hz, hover. The real-aircraft column applies the 42 Hz gyro low-pass filter recorded in the physical log; the calm-PID SITL result uses its logged 20 Hz filter. . . . .                                                            | 36 |
| 9  | INDI parameters, as flown ( <code>sim/indi.param</code> ). . . . .                                                                                                                                                                                                                               | 55 |
| 10 | Other settings referred to in the text, as flown ( <code>sim/stock_pid.param</code> ). . . . .                                                                                                                                                                                                   | 56 |

# LIST OF FIGURES

|   |                                                                                    |    |
|---|------------------------------------------------------------------------------------|----|
| 1 | Overall project workflow, top-down design for prototyping and validation . . . . . | 17 |
|---|------------------------------------------------------------------------------------|----|

|    |                                                                                                                                                                                                                                                                                                                           |    |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2  | CAD model of the twin-boom quadplane configuration: four VTOL lift motors on the booms, a forward pusher motor, and a fixed wing with an independent tail on each boom. . . . .                                                                                                                                           | 18 |
| 3  | Assembled airframe secured to a bench stand for ground testing. . .                                                                                                                                                                                                                                                       | 19 |
| 4  | Purpose: establish the operating envelope of the only physical-flight dataset before using it for validation. The flight is predominantly in hover mode; the short fixed-wing interval remains below 6 m/s. This plot is the evidence for excluding real transition and cruise validation from the present study. . . . . | 28 |
| 5  | Purpose: show representative PID rate tracking in the real and simulated hover records. The panels are deliberately labelled as an unmatched descriptive comparison; their RMS values must not be read as an A/B controller result. . . . .                                                                               | 29 |
| 6  | Simulated PID trajectory tracking versus the desired mission path, 3D view (left) and top-down projection (right), under a 5 m/s wind at 19 m/s cruise airspeed. . . . .                                                                                                                                                  | 30 |
| 7  | OpenVSP-derived aerodynamic performance of the full airframe: lift-to-drag ratio and total pitching-moment coefficient versus angle of attack $\alpha$ (deg). . . . .                                                                                                                                                     | 31 |
| 8  | OpenVSP surface pressure coefficient and local flow-velocity vectors near the fuselage/wing junction, used to sanity-check the centre-of-pressure location. . . . .                                                                                                                                                       | 31 |
| 9  | Purpose: show where the simulated-aircraft inversion gain comes from and how authority is handed from lift motors to aerodynamic surfaces. The fit is from the dedicated PID sweep; the time history is from the calm INDI mission. These values are not hardware identification results. . . . .                         | 35 |
| 10 | Purpose: quantify the differentiated-gyro environment that enters the INDI inversion. The shaded 25 Hz to 90 Hz band is integrated in Table 8. The result validates only the sensor-noise mismatch; it does not validate rigid-body or aerodynamic dynamics. . . . .                                                      | 37 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 11 | Purpose: expose the calm forward transition as a time history. The shaded interval shows the simultaneous motor/surface regime; the lower panels report the resulting roll-rate response and normalised commands. . . . .                                                                                                                                                                                                                                                                                                                                                                    | 40 |
| 12 | Purpose: test the INDI law on its own terms by comparing requested and measured roll angular acceleration below 5 Hz. Statistics use every contiguous phase window; the traces show one representative window. Weak transition correlation prevents a strong tracking claim.                                                                                                                                                                                                                                                                                                                 | 41 |
| 13 | Purpose: compare high-frequency angular-acceleration content under PID and INDI using a signal logged under both controllers. The paired calm missions are shown by flight phase. Shared peaks are treated as plant/model content rather than automatically attributed to INDI. . . . .                                                                                                                                                                                                                                                                                                      | 42 |
| 14 | Purpose: test whether the calm roll results are authority-limited. Neither controller approaches the normalised $\pm 1$ command limit in hover or transition. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                        | 43 |
| 15 | Paired 6 m/s wind experiment: roll and pitch tracking-error RMS by flight phase (upper row), and baseline-subtracted peak error in the five-second window after each scheduled one-second moment pulse (lower row). Points are means of detected pulses, normally over three seeds; whiskers show their minimum and maximum. A few take-off roll markers were not resolved by the lower-rate servo log, so their sample sizes are smaller (one or two; see the per-pulse CSV). Pulse windows are named by their trigger, which can differ from the actual flight phase at injection. . . . . | 44 |
| 16 | Paired 9 m/s wind experiment with the same definitions as Fig. 15. Three seed-paired, full-physics missions were completed per controller; one take-off roll marker was missed in INDI and the cruise roll-pulse result includes assisted flight, not a purely wing-borne test.                                                                                                                                                                                                                                                                                                              | 45 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                               |    |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 17 | Primary logged IMU spectra in manual real hover, a noise-free simulated square-mission landing hover and an independently flown simulated landing hover with calibrated sensor noise. Shading indicates the 20 Hz to 120 Hz band integrated for the six-axis RMS comparison. Unlike real motor vibration, the injected sensor noise is Gaussian; spectral shape and pitch-gyro power remain mismatched. . . . | 46 |
| 18 | First 3D-printed structural iteration of the wing, boom, and tail assembly, prior to final assembly and avionics integration. . . . .                                                                                                                                                                                                                                                                         | 54 |

# Abstract

This thesis investigates attitude control for a vertical take-off and landing (VTOL) quadplane unmanned aerial vehicle (UAV) that combines four vertically oriented lift motors with a fixed-wing airframe and a forward pusher propulsion unit. A physical prototype has been built and flown under the stock ArduPilot cascaded PID controller, but the Incremental Nonlinear Dynamic Inversion (INDI) rate-loop law developed here has been exercised only in Gazebo/ArduPilot software-in-the-loop (SITL) simulation.

Because simulation-model fidelity cannot be assumed, the Gazebo model must be validated against real PID flight logs before any simulated PID-versus-INDI comparison can be trusted. The work reviews nonlinear dynamic inversion and its incremental form, documents the full control chain from IMU measurement through EKF state estimation and the ArduPilot PID loops, derives the INDI rate-loop law with a synchronization correction, and describes the aerodynamic and propeller models.

Six matched SITL missions were completed under PID and INDI in calm, 6 m/s, and 9 m/s crosswind conditions. Activity flags confirm that INDI controlled lift-motor roll and pitch throughout hover and transition and controlled the aerodynamic surfaces throughout transition and forward flight. The results are mixed rather than uniformly favourable: at 6 m/s, INDI reduced transition roll-attitude RMS error from  $1.71^\circ$  to  $0.55^\circ$ , whereas at 9 m/s it increased that error from  $0.70^\circ$  to  $2.45^\circ$  and produced  $11.07^\circ/\text{s}$  roll-rate RMS error. The real-aircraft log cannot validate this transition result because it contains no measured airspeed and only 8.4 s of low-speed fixed-wing operation. It does quantify the principal transfer risk: real filtered roll-axis angular-acceleration noise is  $7.13 \text{ rad/s}^2$  over 25 Hz to 90 Hz, 47 times the calm-SITL value. The implementation is therefore demonstrated in simulation, but neither real-flight validity nor a general INDI advantage is claimed.

**Keywords:** quadplane, VTOL, UAV, INDI, PID, SITL.

# 1 Introduction

## 1.1 Motivation

Aircraft that can take off and land vertically while also travelling efficiently in forward flight are useful for surveying, inspection, mapping, and other missions where a runway is unavailable [source]. A conventional multirotor drone could hover and maneuver precisely, but its propellers need to continuously generate downward thrust to support the aircraft, causing high energy consumption. On the other hand, a fixed-wing aircraft is efficient forward flight, but it typically requires a runway or launching system [source]. A vertical take-off and landing (VTOL) vehicle combines these properties in one system. Currently, it VTOL vehicle can be categorized in to 3 type: Separate lift-cruise system, Tilt-rotors, tilt-wing and We will focus on the quadplane configuration because beyond combining the vertical lift capability of a multirotor with the efficient forward flight of a fixed-wing In this particular project, we look at the effectiveness of the input of the quadplane

The combination is not simply the addition of a quadcopter and an aeroplane. In hover, the vehicle is controlled mainly through the independent thrust of the lift motors. In cruise, the wing produces most of the vertical force and the aircraft is controlled through aerodynamic surfaces and forward propulsion. During transition, both descriptions are active at the same time, and the controller must handle changes in pressure, actuator authority as lift is being created as the vehicle gains speed, trim, coupling, and available control margin (Section 3.3.3).

## 1.2 Research problem

A quadplane combines vertical lift rotors with a winged airframe. While this layout eliminates the need for a runway, it introduces severe control challenges:

1. **Underactuated horizontal motion in hover:** In hover, the vehicle has no horizontal thrusters. Horizontal position and velocity must be controlled by tilting the entire aircraft (roll and pitch) to point the lift vector. Any error or delay in inner-loop attitude tracking immediately degrades outer-loop trajectory tracking.
2. **Complex transition corridor:** During transition between hover and cruise, aerodynamic forces build rapidly over the wings and control surfaces, while

the effective airflow through the lift rotors changes continuously. The control authority shifts from differential thrust of the rotors to aerodynamic surfaces.

3. **Limitations of linear PID:** The conventional cascaded PID controller used in many multirotors relies on fixed gains tuned for a narrow operating regime. It struggles to maintain optimal damping and tracking across the wide dynamic shifts of transition as the nature of the vehicle’s dynamics changes from hover to forward flight. It could be mitigated by using 2 set of PIDs controller

Three properties of ArduPilot motivated building on it rather than developing a custom flight-control stack from first principles:

1. **A validated state estimator.** ArduPilot’s EKF3 fuses IMU, GPS, barometer, and compass measurements into an attitude quaternion, angular rate, position, velocity, and sensor-bias estimate at the rate a rate-loop controller needs. Re-deriving and validating a comparable extended Kalman filter from scratch was judged out of scope for a project whose research question is the rate-loop control law, not state estimation; using ArduPilot’s estimator also lets the INDI controller consume exactly the same estimated states as the PID baseline it is compared against, which is itself a precondition for a fair comparison (Section 4.5).
2. **A proven safety fallback.** Because the INDI controller is implemented as a subclass that overrides only the rate-controller method (Section 4.1), every safety, arming, failsafe, and mode-transition mechanism ArduPilot already provides remains active underneath the new controller. This is what allows the INDI code to run on the platform at all before it has been independently proven, and it is central to the discussion in Section 5.11 of what a smooth first flight does and does not demonstrate.
3. **Resource efficiency.** A single-author academic project cannot realistically re-implement flight-mode logic, a mixer, a logging system, and a parameter system to the reliability level ArduPilot already has, in addition to developing and testing a new control law. Building the new controller as an additional, switchable path inside ArduPilot’s existing attitude-controller class hierarchy concentrates the project’s effort on the actual research contribution.

In simulation, ArduPilot runs as Software-In-The-Loop (SITL): the same firmware source is compiled for a desktop target instead of the flight-controller microcon-

troller, so the exact rate-loop code under test, INDI included, runs unmodified, with MAVLink standing in for the serial/CAN links a real autopilot would use [1]. SITL by itself only exercises the software against a simplified internal physics model; Gazebo is used here as the physics backend instead because ArduPilot maintains an official Gazebo plugin bridge with published conventions for motors, control surfaces, and sensors [2, 3], so the quadplane’s rigid-body and aerodynamic model (Section 5.3) can be built once in an actively maintained, general-purpose robotics simulator with broad community support, rather than a bespoke internal one. This support is also why the sim-to-real verification strategy of Section 4.5 is feasible at all: the same DataFlash log format and message set are produced whether the airframe is real or simulated, so real and simulated logs can be compared channel-for-channel with the same analysis pipeline.

One of the approaches to address these issues is using a flexible controller to consider both the Incremental Nonlinear Dynamic Inversion (INDI), which uses sensor feedback of angular accelerations rather than relying on a complex, global aerodynamic model. However, deploying INDI on a physical aircraft requires solving two primary problems:

- **Scope and architecture:** High-level navigation and state estimation are already mature in open-source autopilots like ArduPilot. The custom control development must be focused precisely where needed—the high-rate inner attitude and vertical acceleration loops—while retaining proven estimator, guidance, and safety fallbacks.
- **Sim-to-real fidelity:** INDI is sensitive to sensor delay, noise filtering, and control-effectiveness errors (particularly in yaw). Testing in simulation alone cannot prove controller stability without first verifying that the simulation reproduces the dynamics seen in physical flight logs.

This thesis addresses these challenges through the following research questions:

1. How can an INDI rate controller be cleanly integrated into the ArduPilot architecture alongside stock state estimation and safety fallbacks?
2. How does sensor filtering and actuator delay matching (synchronization) affect the practical stability of the INDI rate loop?
3. What is required to validate the Gazebo/SITL simulation against real PID flight logs before flight-testing a new controller?

### 1.3 Scope and thesis organisation

The work focuses specifically on inner-loop attitude rate and vertical acceleration control. High-level path planning, navigation, and state estimation (EKF3) are provided by ArduPilot and are not redesigned.

Section 3 reviews rigid-body kinematics, cascaded PID, NDI, and the derivation and synchronization of INDI. Section 4 describes the quadplane platform, the software architecture separating ArduPilot functions from custom INDI code, the simulation environment, and the verification methodology. Section 5 presents simulation results, evaluates controller activity, and discusses limitations. Section 6 concludes and outlines future flight-testing steps.

## 2 Objectives

The primary objective of this thesis is to develop, integrate, and evaluate an Incremental Nonlinear Dynamic Inversion (INDI) attitude control law for a quadplane UAV. The specific aims are:

1. **Define the control hierarchy:** Clearly delineate between subsystems managed by ArduPilot (state estimation, outer attitude kinematics, waypoint navigation, and safety logic) and the custom inner-loop INDI controllers (attitude rate and vertical acceleration augmentation).
2. **Derive and implement synchronized INDI:** Formulate the INDI rate-loop, incorporating delay-matching to prevent phase-lag instability from sensor filtering.
3. **Integrate with ArduPilot flight stack:** Attempt to embed the INDI control law into the ArduPilot QuadPlane architecture in the most inner loop of the attitude control hierarchy.
4. **Establish a verified simulation framework:** Build a semi-accurate model for Gazebo simulation, then establish the baseline needed to validate simulation fidelity against real PID flight data.

## 3 Literature Review

### 3.1 Fundamentals of aircraft and multirotor attitude control

Two established control traditions meet inside a quadplane, and both use the same underlying rigid-body equations but very different actuation and, historically, very different design tools.

### 3.2 Reference frames and the six-degree-of-freedom rigid-body model

Every control law derived later in this chapter, and every equation implemented in simulation in Section 4, is written in terms of the same two reference frames and the same twelve-state rigid-body model, following the standard treatment for small unmanned aircraft [4] which is the main reference for the fundamental principles used for the dynamics and the kinematics.

#### 3.2.1 Reference frames

Two frames are used throughout. The inertial frame  $\mathcal{F}^i$  is fixed to the earth with North-East-Down (NED) axes, and is the frame in which position is naturally defined. The body frame  $\mathcal{F}^b$  is fixed to the airframe with origin at the centre of gravity and forward-right-down (FRD) axes (note: following standard aeronautical conventions where the  $z$ -axis points downward, which differs from the forward-left-up [FLU] orientation typically used in (put the introduction the simulation tool used here the exact mentioned in whatever section) simulation tools); it is the frame in which velocity, angular rate, forces, and moments are naturally expressed, because the inertia matrix  $J$  is constant there and the IMU measures specific force and angular rate directly in it. The two frames are related by the 3-2-1 Euler sequence – yaw  $\psi$  about the inertial  $z$ -axis, then pitch  $\theta$  about the resulting  $y$ -axis, then roll  $\phi$  about the resulting  $x$ -axis – giving the rotation matrix  $R_i^b(\phi, \theta, \psi)$  used to convert vectors between  $\mathcal{F}^i$  and  $\mathcal{F}^b$  [4].

### 3.2.2 Twelve states, six degrees of freedom

A rigid body free to move in three dimensions has six *degrees of freedom* (DOF): three translational (motion along the body  $x$ ,  $y$ , and  $z$  axes) and three rotational (rotation about those same three axes). Each of these six degrees of freedom is described in a first-order state-space model by a position-like variable and its own rate-like variable, which is why the model has twelve states rather than six:

- **Translational (6 states):** position  $(p_n, p_e, p_d)$  in  $\mathcal{F}^i$ , paired with body-frame velocity  $\mathbf{v} = (u, v, w)$  in  $\mathcal{F}^b$ .
- **Rotational (6 states):** attitude, given by the Euler angles  $(\phi, \theta, \psi)$ , paired with body-frame angular rate  $\boldsymbol{\omega} = (p, q, r)$  in  $\mathcal{F}^b$ .

This full state vector  $\mathbf{x} = [p_n, p_e, p_d, u, v, w, \phi, \theta, \psi, p, q, r]^T$  is the twelve-state model used in [4]. The kinematic pairing is

$$\dot{\mathbf{p}}_i = R_b^i(\phi, \theta, \psi) \mathbf{v}, \quad \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi / \cos \theta & \cos \phi / \cos \theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}, \quad (1)$$

where  $\mathbf{p}_i = (p_n, p_e, p_d)^T$  and the Euler-rate transform is singular at  $\theta = \pm 90^\circ$  [4].

Section 3.2.3 below gives the corresponding dynamic equations, i.e. the force/moment-driven rate of change of  $\mathbf{v}$  and  $\boldsymbol{\omega}$ .

### 3.2.3 Rigid-body dynamics

Newton's second law, applied in the rotating body frame with the transport theorem  $\frac{d}{dt}\big|_i = \frac{d}{dt}\big|_b + \boldsymbol{\omega} \times$ , gives the translational and rotational equations of motion

$$m(\dot{\mathbf{v}} + \boldsymbol{\omega} \times \mathbf{v}) = \mathbf{F}_a + \mathbf{F}_p + \mathbf{F}_g, \quad (2)$$

$$J\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (J\boldsymbol{\omega}) = \boldsymbol{\tau}_a + \boldsymbol{\tau}_p, \quad (3)$$

where  $\mathbf{F}_a, \mathbf{F}_p, \mathbf{F}_g$  and  $\boldsymbol{\tau}_a, \boldsymbol{\tau}_p$  are the aerodynamic, propulsive, and gravitational contributions, and  $\mathbf{v}, \boldsymbol{\omega}$  are body-frame velocity and angular rate as defined above [4]. The  $\boldsymbol{\omega} \times \mathbf{v}$  term in Eq. (2) is the centripetal contribution that a rotating body frame have in  $F = ma$ ; it is easy to drop by mistake because it vanishes identically at hover ( $\boldsymbol{\omega} \approx 0$ ) and so is not exposed by hover-only testing. Together,

Eqs. (1)–(3) form the rigid-body model implemented across the Gazebo plugins (Section 4.1); they are also the specific instance of Eq. (4) that Sections 3.4–3.5 below are derived for.

### 3.2.4 How many variables control the body frame?

Attitude, taken by itself, has three rotational degrees of freedom, and Eq. (3) needs in general three independent moments – one about each body axis – to control it fully. This is exactly the three-axis roll/pitch/yaw command used by the rate loop implemented in this thesis (Eq. (13)): full attitude control is a three-variable problem, not a two-variable one.

A narrower, frequently conflated claim concerns not attitude but *translational* force. A multirotor’s only direct translational actuator is a scalar collective thrust along the body  $z$ -axis; there is no separate horizontal-thrust actuator. To redirect that thrust horizontally – for example to correct horizontal position or track a velocity command – the vehicle must tilt its thrust axis, and doing so only requires two of the three attitude angles, roll and pitch: they set the *direction* of the body  $z$ -axis, which has only two degrees of freedom (a direction in three-dimensional space is a point on a sphere, parametrised by two angles), while yaw rotates the body about that same axis without changing its direction at all, and is therefore decoupled from thrust direction [5]. This is why hovering multirotor control architectures command all three attitude angles, but use only two of them,  $(\phi, \theta)$ , as intermediate “virtual controls” generated by an outer position/velocity loop, while the third,  $\psi$ , is set independently, typically to hold or track heading [4, 5]. The body frame’s *orientation* therefore still needs three variables to be fully controlled; it is specifically the redirection of translational *thrust* that needs only two of them, with the third left free.

### 3.2.5 Scope of custom controller

To understand the control structure, it is essential to distinguish between what ArduPilot manages and what this thesis develops.

A quadplane has twelve rigid-body states: position  $(p_n, p_e, p_d)$ , body velocity  $(u, v, w)$ , Euler angles  $(\phi, \theta, \psi)$ , and body angular rates  $(p, q, r)$ . In this project, the control responsibility is divided as follows:

- **Controlled by stock ArduPilot (unmodified):**
  - **Position and trajectory:** Horizontal position  $(p_n, p_e)$  and horizontal velocities  $(u, v)$  are controlled by ArduPilot’s navigation and position controller in VTOL modes, and by TECS in forward flight (Section 3.3.2).
  - **Outer-loop attitude kinematics:** ArduPilot converts attitude targets into desired body angular rates  $\boldsymbol{\omega}_{\text{des}} = (p_{\text{des}}, q_{\text{des}}, r_{\text{des}})$ . This kinematic mapping is preserved without change.
  - **Actuator allocation and safety:** The motor mixer, control surface mapping, saturation limits, motor spooling, and failsafe logic are entirely provided by ArduPilot.
- **Developed in this thesis (custom inner-loop control):**
  - **Attitude rate loop  $(p, q, r)$ :** The custom contribution replaces ArduPilot’s inner PID rate controller with an Incremental Nonlinear Dynamic Inversion (INDI) rate law in hover (Section 4.3) and transition (Section 4.4). It computes the virtual control moments needed to drive  $(p, q, r)$  toward  $\boldsymbol{\omega}_{\text{des}}$ .
  - **Vertical axis augmentation  $(p_d, w)$ :** ArduPilot’s vertical position and velocity loops are retained, while a bounded acceleration-feedback trim is added to the throttle command (Section 4.3) to improve vertical disturbance rejection.

Why is inner-loop attitude control so critical in a quadplane? In hover, the vehicle acts like a multirotor. It is underactuated: it has no direct horizontal thrusters. To move forward, backward, or sideways, it must tilt its body (roll and pitch) to point the lift-motor thrust vector in the desired direction. If attitude rate tracking is sluggish or unstable, horizontal position control degrades immediately. Furthermore, during transition to forward flight, dynamic pressure builds over the wings and control surfaces while lift-motor airflow changes dramatically. The inner loop must maintain stable attitude and damping across these aerodynamic shifts so the aircraft stays within its safe flight envelope.

### 3.3 Attitude control architectures across vehicle classes

#### 3.3.1 Multirotor rate/attitude control

A multirotor is actuated only through the thrust of  $n$  fixed-orientation rotors. Because a change in one rotor’s thrust changes body torque almost instantaneously and with a nearly constant, well-conditioned linear map near hover, multirotor flight controllers converge on the same nested-loop structure: an outer attitude loop converts an orientation error into a desired body rate, and an inner, much faster rate loop converts a rate error into a torque/thrust demand that a linear mixer distributes to the rotors. Lombaerts et al. [6] and Zhou et al. [7] describe and use this same cascade even where the inner loop itself is replaced by INDI, which indicates that the cascade is a property of the control problem’s structure rather than of the particular inner-loop law chosen.

#### 3.3.2 Fixed-wing attitude and energy control

A fixed-wing aircraft controls attitude through aerodynamic surfaces whose effectiveness scales with dynamic pressure  $\bar{q} = \frac{1}{2}\rho V^2$ , and it does not have direct, independent control over vertical force the way a multirotor does: climb rate and airspeed are coupled through the aircraft’s total energy rather than through two separate actuators. ArduPlane’s altitude/airspeed controller therefore does not treat climb rate and speed as two independent PID loops; it uses a Total Energy Control System (TECS) architecture, which separates the throttle and elevator commands along energy-rate and energy-distribution axes instead [1, 8]. This distinction matters for a quadplane because the transition phase is exactly where the vehicle moves from multirotor-style direct force control to fixed-wing-style energy-coupled control.

#### 3.3.3 The quadplane hybrid case and the transition corridor

A quadplane’s four lift rotors give it the multirotor property of direct, decoupled roll/pitch/yaw/thrust control at zero airspeed. As airspeed increases, the wing begins to carry weight and the control surfaces begin to carry authority, so the vehicle’s usable envelope during the changeover is bounded on both sides: below some airspeed the wing has not yet taken over enough lift or control authority to fly like an aeroplane, and above some airspeed the lift rotors are unnecessary or

may interfere aerodynamically. The band of airspeed, and the schedule of control blending, over which the vehicle can safely operate in this mixed regime is what this thesis calls the *transition corridor*, following the blending-coefficient formulation  $\eta_{\text{aero} \rightarrow \text{hover}}$  that Lombaerts et al. [6] use to schedule aerodynamic versus hover-regime force models across the same speed range, and the explicitly separate hover and fixed-wing INDI control laws that Zhou et al. [7] designed for a hybrid quad-plane UAV specifically because a single control law was not judged adequate across the whole corridor. The hover-end controller of Section 4.3 does not by itself cover this corridor. Section 4.4 describes a second, separately-scheduled INDI law added for the transition and fixed-wing regime, following the same separate-hover/fixed-wing-law structure Zhou et al. [7] used rather than a single blended controller, with the flight validation reported in Section 5.8.

### 3.4 Nonlinear dynamic inversion: general formulation and its central problem

Consider a nonlinear, input-affine system

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{G}(\mathbf{x})\mathbf{u}, \quad \mathbf{y} = \mathbf{h}(\mathbf{x}), \quad (4)$$

with state  $\mathbf{x}$ , control input  $\mathbf{u}$ , and output  $\mathbf{y}$ . Differentiating an output  $y_i$  repeatedly with respect to time eventually produces a derivative  $y_i^{(r_i)}$  that depends explicitly on  $\mathbf{u}$ ; the smallest such order  $r_i$  is the *relative degree* of that output [6, 9]. Collecting all outputs at their relative degree gives

$$\mathbf{y}^{(r)}(\mathbf{x}) = \mathbf{l}(\mathbf{x}) + \mathbf{M}(\mathbf{x})\mathbf{u}, \quad (5)$$

where  $\mathbf{l}$  collects the state-dependent terms and  $\mathbf{M}$  is the control-effectiveness matrix [6]. If  $\mathbf{M}$  is invertible (or has a well-posed generalised inverse for an overactuated system), Eq. (5) can be solved for the command that would produce a desired output derivative  $\boldsymbol{\nu} = \mathbf{y}_{\text{des}}^{(r)}$ :

$$\mathbf{u}_{\text{cmd}} = \mathbf{M}^{-1}(\mathbf{x}) [\boldsymbol{\nu} - \mathbf{l}(\mathbf{x})]. \quad (6)$$

Substituting Eq. (6) back into Eq. (5) cancels the nonlinearity exactly in the ideal case and leaves  $\mathbf{y}^{(r)} = \boldsymbol{\nu}$ : the aircraft, together with its own inverse, behaves like a bank of pure integrators driven by the new, *virtual* input  $\boldsymbol{\nu}$  [6]. For rotational control this is exactly the rigid-body form of Eq. (3) (Section 3.2):  $\mathbf{l}(\mathbf{x}) = \mathbf{J}^{-1}(-\boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega})$  and  $\mathbf{M}(\mathbf{x}) = \mathbf{J}^{-1}\mathbf{M}_{\text{CA}}(\mathbf{x})$ , where  $\mathbf{M}_{\text{CA}}$  maps actuator commands to moments [6].

This is an elegant result with one demanding practical requirement: Eq. (6) needs  $\mathbf{l}(\mathbf{x})$  and  $\mathbf{M}(\mathbf{x})$  – the entire nonlinear aerodynamic and inertial model – known accurately at the current state, for every state the aircraft will visit. Early NDI flight-control work on high-performance aircraft found that when this model knowledge was uncertain the closed loop could lose robustness [10, 11]. This dependence is the specific problem that motivated the incremental reformulation used in this thesis (Section 3.5): as Steinert et al. [9] frame it, INDI emerged “with the characteristics of being less model-dependent” precisely because “NDI is a model-based approach that depends on the quality of the mathematical model utilized,” and prior work “encountered robustness issues when the model’s uncertainty was too large.” For a quadplane whose wing and propeller aerodynamics are only approximately known (Section 5.3), this is not a hypothetical concern: it is the primary reason full NDI was not judged a viable candidate for this project.

## 3.5 Incremental nonlinear dynamic inversion

### 3.5.1 Origins and the Taylor-series derivation

The incremental idea was first flight-tested by Smith and Berry [12] using a “simplified nonlinear dynamic inversion” law that Smith [13] had derived directly from the aircraft’s rotational dynamics rather than from the general Lie-derivative form of Section 3.4. Bacon and Ostroff [14] generalised this to a first-order Taylor-series derivation that also covers input non-affine systems, and this is the derivation reproduced below because it matches how the rate loop is actually implemented (Section 4.3). Sieberling et al. [15] later revisited Bacon and Ostroff’s law, named it Incremental Nonlinear Dynamic Inversion, and examined its sensitivity to time delay in the angular-acceleration feedback – a sensitivity that turned out to be central to this project’s own implementation history (Section 5.12.1).

Consider the system of Eq. (4) written for one differentiation as  $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x}, \mathbf{u})$ .

A first-order Taylor expansion of  $\mathbf{F}$  around the operating point one sample in the past,  $(\mathbf{x}_0, \mathbf{u}_0) = (\mathbf{x}(t - T_s), \mathbf{u}(t - T_s))$ , gives

$$\dot{\mathbf{x}} \approx \dot{\mathbf{x}}_0 + \left. \frac{\partial \mathbf{F}}{\partial \mathbf{x}} \right|_0 (\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial \mathbf{F}}{\partial \mathbf{u}} \right|_0 (\mathbf{u} - \mathbf{u}_0), \quad (7)$$

where  $T_s$  is the control-loop sample time [9, 14]. Under the *time-scale separation* assumption – that the state changes negligibly over one sample compared with the control input,  $\mathbf{x} - \mathbf{x}_0 \ll \mathbf{u} - \mathbf{u}_0$ , which holds when the control-update rate is fast relative to the rigid-body dynamics and the actuators are fast and close to ideal [14, 15] – the state-dependent term is dropped, leaving

$$\dot{\mathbf{x}} \approx \dot{\mathbf{x}}_0 + \mathbf{G}(\mathbf{x}_0) \Delta \mathbf{u}, \quad \Delta \mathbf{u} = \mathbf{u} - \mathbf{u}_0. \quad (8)$$

Setting the pseudo control  $\boldsymbol{\nu} = \dot{\mathbf{x}}$  and inverting Eq. (8) for the required increment gives the incremental control law

$$\Delta \mathbf{u} = \mathbf{G}(\hat{\mathbf{x}}_0)^{-1} [\boldsymbol{\nu} - \hat{\dot{\mathbf{x}}}_0], \quad \mathbf{u} = \hat{\mathbf{u}}_0 + \Delta \mathbf{u}, \quad (9)$$

where  $(\hat{\cdot})_0$  denotes a measured or estimated quantity from the previous sample [9, 14]. Comparing Eq. (9) with the NDI law of Eq. (6), the crucial difference is that only the control-effectiveness term  $\mathbf{G}(\mathbf{x}_0)$  needs to be modelled; the state-dependent term  $\mathbf{I}(\mathbf{x})$  has been replaced entirely by the measured current state derivative  $\hat{\dot{\mathbf{x}}}_0$  [9]. This is why INDI is described as a *sensor-based* rather than *model-based* approach: an equivalent derivation obtained by solving the plant equation directly for its unknown state-dependent term and substituting the measured output derivative in its place gives the identical law without invoking a Taylor approximation at all, for an input-affine, relative-degree-one output [9].

Applied to the rotational dynamics of Eq. (3) with  $\mathbf{x} = \boldsymbol{\omega}$  and pseudo control  $\boldsymbol{\nu} = \dot{\boldsymbol{\omega}}_{\text{des}}$ , Eq. (9) becomes the rate-loop law used later in Eqs. (13)–(14), with  $\mathbf{G}_\omega$  playing the role of  $\mathbf{G}(\hat{\mathbf{x}}_0)$ : a control-effectiveness matrix mapping a change in normalised roll/pitch/yaw command to a change in angular acceleration.

Two consequences of this derivation matter for everything that follows. First, INDI replaces model dependence with a dependence on the *measurement* quality of  $\hat{\dot{\mathbf{x}}}_0$  and  $\hat{\mathbf{u}}_0$ , which introduces the estimation and synchronization problem of Section 3.6. Second, the time-scale separation assumption is only as good as the

control-update rate and actuator speed relative to the rigid-body dynamics [15]; this project does not independently verify that assumption for the quadplane’s motors and lists it as a limitation (Section 5.12).

### 3.5.2 Why INDI over PID, gain-scheduled NDI, or MPC

Three properties, summarised across both parts of the Steinert et al. survey and demonstrated directly by Lombaerts et al. [6] for a hovering tiltrotor eVTOL, motivate choosing INDI over the alternatives available for this project:

- **Versus a single set of PID gains.** A cascaded PID controller, as used in the ArduPilot baseline (Section 4.3), is tuned around one operating condition; a quadplane’s dynamics change enough between hover and cruise that a single gain set is not expected to remain well damped across the envelope without gain scheduling or blending. Lombaerts et al. [6] note that both NDI and INDI avoid this specifically because “the control laws are split into a model dependent part and a model independent part,” so “gain scheduling is not needed”: the linear reference-model and error-controller part of the loop can be tuned once, independent of airframe and flight condition, while only the effectiveness matrix, not the loop gains, needs to track the operating point.
- **Versus full (non-incremental) NDI.** As derived in Section 3.4, full NDI needs the complete nonlinear model  $\mathbf{l}(\mathbf{x})$  at every operating point; INDI needs only the local effectiveness  $\mathbf{G}(\mathbf{x}_0)$ , replacing the rest with a measurement [9]. Steinert et al. [9] further note that, because the measurement of  $\dot{\mathbf{x}}_0$  already contains whatever disturbance acted on the vehicle, INDI rejects disturbances “one integration level earlier” than NDI does – a property with no PID or full-NDI analogue.
- **Versus model predictive control (MPC).** MPC solves a constrained optimisation over a prediction horizon at every control step, which requires both an accurate predictive model over that horizon and a computational budget that a small flight-control processor may not comfortably provide at the several-hundred-hertz rate an attitude rate loop needs; it also reintroduces the full-model dependence that motivated moving away from NDI in the first place. None of the three primary sources reviewed for this thesis compare INDI against MPC directly, so this point is a reasoned engineering judgement

rather than a literature-backed comparison, and is presented as such.

None of this makes INDI automatically better: its own dependence on the effectiveness matrix and on the quality and timing of the acceleration measurement is the subject of the rest of this chapter and of the implementation problems reported in Section 5.12.

### 3.6 Estimating angular acceleration and the synchronization problem

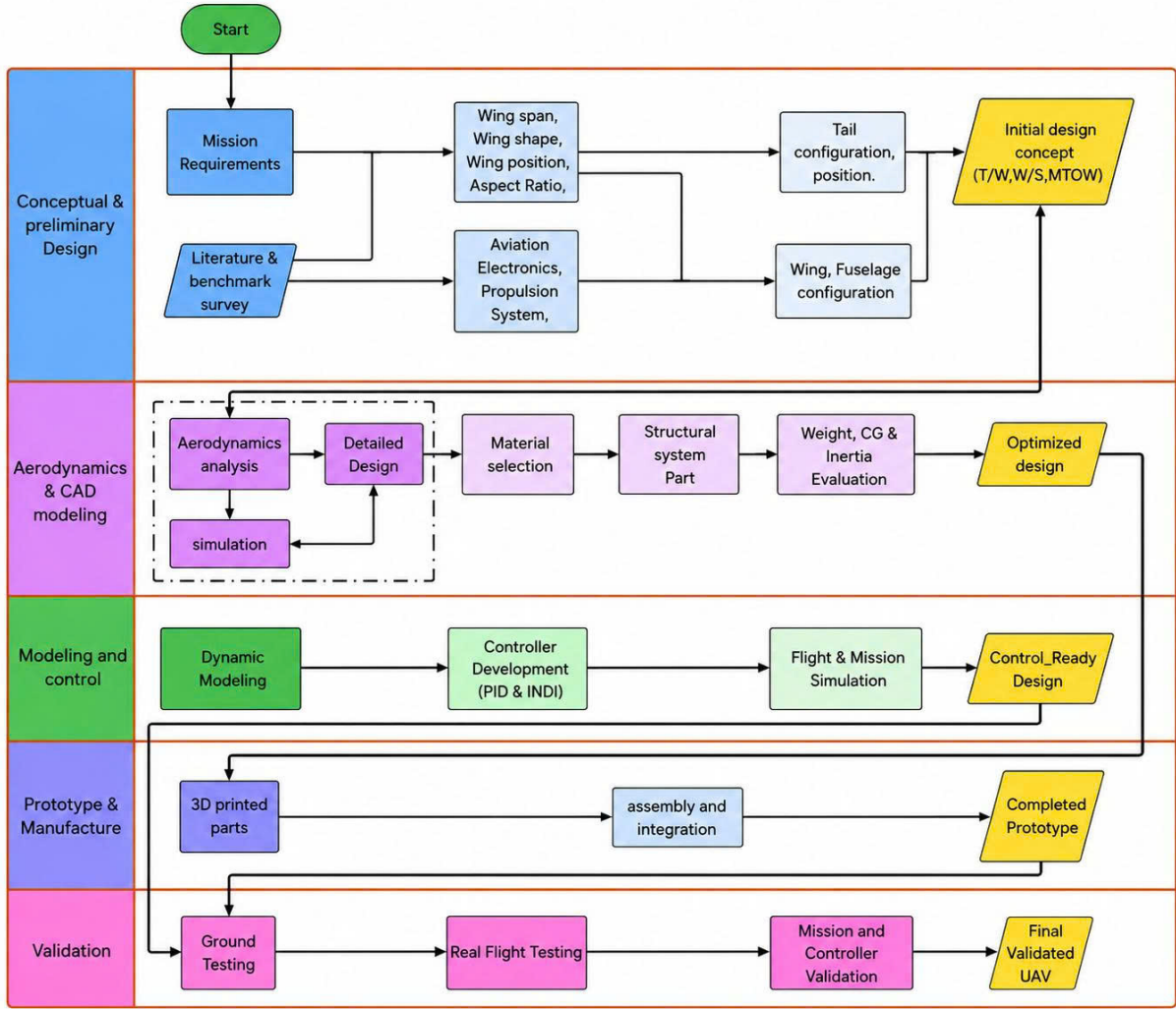
Aircraft are rarely equipped with angular-acceleration sensors, so  $\dot{\omega}$  must be estimated from the measured angular rate  $\omega$ , most commonly by a backward difference [16]. Because numerical differentiation amplifies high-frequency sensor noise, essentially every practical INDI implementation low-pass filters this estimate [16]; the filter attenuates noise but necessarily introduces phase lag.

Smeur et al. [17] showed that this lag has a specific, quantifiable consequence: if the acceleration estimate is filtered but the actuator-command feedback used in the incremental update is not delayed by an equivalent amount, the closed-loop relation between the commanded and true pseudo control no longer matches the actuator dynamics, degrading phase margin. Their fix, which Steinert et al. [16] call *synchronization*, is to pass the command feedback  $\mathbf{u}_0$  through the same filter used for the acceleration estimate before it is used in the incremental update, so that both signals entering the control law are delayed by the same amount. Under this synchronization, and given accurate effectiveness knowledge, the closed-loop transfer function from commanded to true pseudo control reduces to the actuator dynamics alone [16]. van 't Veld et al. [18] independently confirmed, using an analytical SISO example and a coupled rate/attitude-loop simulation, that INDI is markedly more sensitive to a delay surplus on the acceleration-feedback path than to an equal surplus on the actuator-feedback path, and that the largest stable region is obtained when the two paths are delayed *equally* – that is, when they are synchronized. Steinert et al. [16] recommend exactly this actuator-feedback synchronization scheme as the appropriate low-complexity choice whenever the system is close to single-input/single-output per axis and the state dynamics are slow relative to the actuator, which is the assumption made for this project's diagonal, hover-only effectiveness matrix (Section 4.3).

Formal stability treatments of INDI, such as the Lyapunov- and perturbation-based analysis of Wang et al. [19], show that the incremental relation’s error term shrinks as the sampling interval shrinks and that INDI’s disturbance-rejection advantage over NDI is provable under stated assumptions on actuator and sensor dynamics – but the same analyses also show that singular perturbations such as actuator dynamics and finite sampling time can make the closed loop’s error bound grow unboundedly for some parameter combinations, which is the formal counterpart of the synchronization problem just described. This synchronization requirement is not a footnote for this project: Section 5.12.1 reports that the first implementation of the rate loop did not include it, and describes the fix that was applied once the omission was identified against this literature.

## 4 Methodology

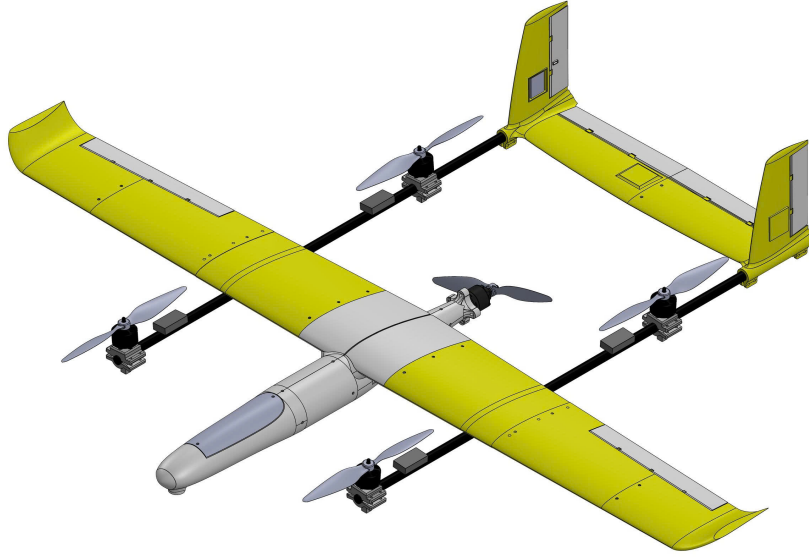
The work followed the steps in Fig. 1. The aircraft was designed and built first. Its aerodynamics were then modelled in OpenVSP and transferred into a Gazebo model. ArduPilot was downloaded and run against that model in simulation. An INDI rate loop was added to ArduPilot and compared with the stock PID loop on the same missions, in the same wind, with the same disturbances. Only the PID controller has flown on the real aircraft, under manual control; the INDI controller has flown in simulation only.



**Figure 1.** Overall project workflow, top-down design for prototyping and validation

## 4.1 Aircraft, autopilot and simulator

The aircraft is a twin-boom quadplane, Fig. 2. A central fuselage and wing carry two tail booms. Each boom holds two lift motors and its own tail. A pusher motor on the fuselage gives thrust for cruise. The lift motors hold the aircraft in hover and give the control moments at low speed; the wing takes over as airspeed builds. The flight is therefore split into hover, transition and fixed-wing cruise (Section 3.3.3). The assembled airframe was tested on a bench stand, Fig. 3.



**Figure 2.** CAD model of the twin-boom quadplane configuration: four VTOL lift motors on the booms, a forward pusher motor, and a fixed wing with an independent tail on each boom.



**Figure 3.** Assembled airframe secured to a bench stand for ground testing.

**Autopilot.** ArduPilot was chosen because it is mature, open-source flight software that already supports quadplanes. It provides, unchanged:

- sensor drivers and an extended Kalman filter that estimates attitude, rates, position and velocity;
- the outer attitude loop, which turns a desired attitude into a desired body rate;
- guidance, navigation, flight modes and the transition logic between hover and wing-borne flight;
- the mixer, which turns roll, pitch, yaw and throttle commands into motor and servo outputs;
- parameter storage and flight logging;
- the stock PID rate loop, which stays in the code as the fallback.

Only the inner body-rate loop is replaced. Everything before it (attitude error, rate target) and after it (mixer, outputs) is stock ArduPilot. Navigation, waypoint following and altitude control are left to ArduPilot and are the same for both controllers. A boot-time switch decides whether the INDI code is loaded at all; with the switch off, the aircraft runs the stock firmware behaviour.

**Simulator.** The same ArduPilot source is compiled for a desktop computer (software-in-the-loop, SITL) and connected to Gazebo, which computes the rigid-body motion and aerodynamic forces. A bridge passes motor and servo commands one way and IMU, barometer, GPS and airspeed data the other. Real and simulated flights produce the same log format, so one analysis pipeline reads both.

**Frames.** The body frame is forward-right-down, as in Section 3.2 and in ArduPilot. Gazebo uses east-north-up world axes, so the bridge converts between the two. A sign error here would reverse the aerodynamic and propulsive terms of Eqs. (2)–(3), and the controller would look unstable when the fault is in the model interface.

## 4.2 Aerodynamic model: from OpenVSP to Gazebo

The airframe geometry was built in OpenVSP [20]. Its vortex-lattice solver swept angle of attack and returned the lift, drag and pitching-moment coefficients

of the whole aircraft, together with the reference area, span and chord. Lift and drag follow

$$L = \frac{1}{2}\rho V^2 S C_L, \quad D = \frac{1}{2}\rho V^2 S C_D. \quad (10)$$

Gazebo does not read an OpenVSP file. Its lift-drag system models each surface as one flat panel described by a handful of numbers, listed in Table 2. The wing, aileron sections, horizontal tail, elevator, vertical fins, rudders and winglets are each one panel, or a left and right pair. Below stall the panel lift coefficient is linear in angle of attack,

$$C_L = C_{L\alpha}(\alpha + \alpha_0) + k_\delta \delta, \quad (11)$$

where  $\delta$  is the deflection of the surface’s control joint, if it has one. Past the stall angle the slope changes to a separate post-stall value. The OpenVSP results must be turned into these numbers by hand; the solver’s own output is not used during flight. Section 5.3 checks how well the flown model matches the OpenVSP sweep.

**Table 2.** Parameters of one Gazebo lift-drag panel, with the main-wing values of the flown model and the matching OpenVSP value where one exists. The SDF tag names are given in Appendix C.

| Symbol                                   | Meaning                                      | Main wing (flown)          | OpenVSP              |
|------------------------------------------|----------------------------------------------|----------------------------|----------------------|
| $S$                                      | panel area                                   | 0.183 m <sup>2</sup>       | 0.204 m <sup>2</sup> |
| $\alpha_0$                               | angle added to the geometric angle of attack | 0.130 rad                  | 0.130 rad            |
| $C_{L\alpha}$                            | lift-curve slope                             | 3.70/rad                   | 5.0/rad              |
| $C_{D\alpha}$                            | drag-curve slope                             | 0.064/rad                  | 0.064/rad            |
| $C_{m\alpha}$                            | pitching-moment slope                        | 0                          | from C               |
| $\alpha_{\text{stall}}$                  | stall angle                                  | 0.339 rad (19.4°)          | not                  |
| $C_{L\alpha,\text{stall}}$               | lift slope after stall                       | −3.85/rad                  | not                  |
| $C_{D\alpha,\text{stall}}$               | drag slope after stall                       | −0.92/rad                  | not                  |
| $\mathbf{r}_{cp}$                        | centre of pressure, body frame               | (0.061, 0, 0.004) m        |                      |
| $\hat{\mathbf{e}}_f, \hat{\mathbf{e}}_u$ | panel forward and upward axes                | + $x$ , + $z$              |                      |
| $k_\delta$                               | lift coefficient per radian of deflection    | — (ailerons, elevator: −5) |                      |

Each propeller is modelled the same way: two small lift-drag panels on a spinning link. ArduPilot commands the rotor speed, and the panels give a thrust that

grows with the square of that speed.

### 4.3 Rate controllers

**PID baseline.** The stock loop compares the desired body rate with the measured rate and outputs a normalised command per axis,

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt}, \quad (12)$$

where  $e$  is the rate error. The mixer turns the three commands and the throttle into motor and servo outputs. The PID runs are the stock firmware with the INDI switch off.

**INDI law.** INDI needs one model number per axis, the control effectiveness  $G$ : the change in angular acceleration per unit change of normalised command,

$$\Delta\dot{\omega} = G \Delta u. \quad (13)$$

Each axis (roll, pitch) then runs the same incremental law every control cycle:

$$\begin{aligned} \nu &= K (\omega_{\text{ref}} - \omega) && \text{desired angular acceleration} \\ \dot{\omega}_f &= H\left(\frac{d\omega}{dt}\right) && \text{measured angular acceleration} \\ u_{0,f} &= H(F_g(A(u_{\text{applied}}))) && \text{effector state, delayed to match} \\ u &= u_{0,f} + \frac{\nu - \dot{\omega}_f}{G} && \text{new command} \end{aligned} \quad (14)$$

The derivative is a backward difference of the gyro rate.  $H$  is a low-pass filter with two equal poles at 12.7 Hz.  $A$  is a model of the effector: a pure delay followed by a first-order lag.  $F_g$  is a copy of the gyro filter that ArduPilot applies before the rate reaches the controller.

**Why the command path is filtered.** The incremental relation of Eq. (8) compares an acceleration with the command that caused it. The measured acceleration has passed through the effector lag, the gyro filter and  $H$ . The command  $u_{\text{applied}}$  is therefore passed through the same three stages,  $A$ ,  $F_g$  and  $H$ , so that both signals arrive at the subtraction with the same delay:

$$u_{0,f} = H F_g A u_{\text{applied}}. \quad (15)$$

Filtering with  $H$  alone was not enough; the loop limit-cycled until all three stages were matched (Section 5.12.1).  $u_{\text{applied}}$  is what the mixer actually sent in the last cycle, recovered from the motor outputs, so saturation and mixer limits are included.

**Effectiveness.** For the lift motors,  $G$  is a fixed value per axis, identified in hover (Section 5.4). For the control surfaces the moment grows with dynamic pressure, so

$$G_s(V) = G_{s,\text{ref}} \left( \frac{V}{V_{\text{ref}}} \right)^2, \quad (16)$$

with  $G_{s,\text{ref}}$  identified at the reference airspeed  $V_{\text{ref}} = 15 \text{ m/s}$ . Yaw stays on the PID in both cases: its effectiveness could not be identified (Section 5.7).

## 4.4 Which effector runs INDI, and when

A quadplane has two sets of effectors for roll and pitch: the lift motors and the control surfaces. ArduPilot already decides, every cycle, which rate loop drives which set. It uses its transition logic: whether the aircraft is in a hover mode, whether the forward transition has finished, and whether the lift motors are currently helping the wing (“assist”). This project does not change that decision. It only replaces the PID with INDI on the set that ArduPilot has put in charge, following the rule below.

```

1  if hover mode, or transition not finished, or lift motors
    assisting:
2      lift motors    -> INDI  (roll, pitch)
3      surfaces       -> stock PID
4  else:                                     # wing-borne flight
5      lift motors    -> off
6      surfaces       -> INDI  (roll, pitch)
7  yaw               -> stock PID, always
8  any axis with G = 0, or motors not spun up -> stock PID

```

**Listing 1.** Effector ownership, evaluated every control cycle.

The rule gives one property by construction: at any instant at most one INDI loop acts on an axis. Two INDI loops acting together would each invert the same acceleration error, and the aircraft would receive twice the intended correction. The cost is that during the transition the surfaces are driven by the PID while

the motors are driven by INDI, so the surface moment is an unmodelled input for the motor INDI loop. INDI measures the resulting acceleration and corrects it at the next step, which is the robustness argument of Section 3.5, but it is not a joint allocation. The internal names of this decision in ArduPilot are listed in Appendix D.

When INDI is not running on an axis, its states follow the PID output, so a later switch to INDI starts from the current command without a jump.

## 4.5 Simulation set-up and flight profiles

Each case runs in its own folder with a fresh autopilot memory, so no setting carries over from the previous run. Every parameter is checked afterwards against the values the autopilot wrote into its log at boot. Each case archives the log, the world file, the parameter file, the console output and a short summary of whether the aircraft armed, transitioned, reached every waypoint and landed. Two missions are flown, Table 3.

**Square circuit.** Vertical take-off to 35 m, a closed square of about 250 m per side, and a vertical landing at the take-off point. One circuit contains every phase: hover, forward transition, wing-borne flight with four 90° turns, back transition, and hover. Because the aircraft flies all four headings, one steady wind is a crosswind, headwind, opposite crosswind and tailwind in turn, all in one log.

**Zigzag.** A lawn-mower pattern to test quick turns: five parallel legs of 500 m, 100 m apart, flown alternately north and south (heading 356° and 176°) at 35 m and 16.1 m/s. Each leg runs 10 m past its end before the turn. The four turnarounds are each two 90° turns 100 m apart, so the aircraft reverses direction in a short distance and the roll axis is loaded far more often than on the square. The plan ends with a landing at the far corner of the pattern, about 765 m from the take-off point.

**Table 3.** Flight profiles. Both are read directly from their mission files, so the desired path in every figure and the flown mission come from the same source.

|          | Square circuit                | Zigzag                                      |
|----------|-------------------------------|---------------------------------------------|
| Legs     | $4 \times 250$ m              | $5 \times 500$ m, 100 m apart               |
| Turns    | $4 \times 90^\circ$           | $4 \times 180^\circ$ (each two $90^\circ$ ) |
| Altitude | 35 m                          | 35 m                                        |
| Landing  | at take-off point             | far end, 765 m away                         |
| Tests    | all phases, all wind headings | repeated quick turns                        |

**Airspeed settings.** A quadplane switches its lift motors on to help the wing (“assist”) whenever it believes the airspeed is too low. In the earlier campaign a setting forced this help on for the whole flight, and the speed limits were above what the aircraft could reach. The “fixed-wing” part of every earlier run was therefore flown with the lift motors running, and no surface controller was ever tested on the wing alone. The speed limits were re-set from the model’s stall speed (Section 5.3):

- stall speed 12 m/s;
- minimum airspeed 14 m/s;
- lift-motor assist only below 13 m/s, as a low-speed safety net, and the forced-assist setting cleared;
- cruise 17 m/s, which the model holds.

## 4.6 Wind and disturbance injection

Wind is set in the Gazebo world file as one steady vector per case, with small turbulence (0.3 m/s in magnitude, 0.02 rad in direction, 0.05 m/s vertically). Three things must hold for the wind to be real, and each failed at some point in this work:

1. *The aerodynamics see the wind.* Each lift-drag panel computes its own air-relative velocity from the world wind.
2. *The wind does not also push the body.* Gazebo’s default wind plugin adds a drag force  $F = m(\mathbf{v}_w - \mathbf{v})$ , about 48 N at cruise for this aircraft, which stops the transition. This force is switched off; the panels already produce the aerodynamic drag.

3. *The autopilot sees the same wind.* The bridge now sends the wind vector to ArduPilot, so its airspeed is air-relative and not ground speed.

The crab angle in crosswind checks that all three hold (Section 5.6).

The disturbance is a torque applied directly to the airframe in Gazebo. Its size is the rolling moment of a gust of the same speed as the wind acting on one half of the wing,  $M = \frac{1}{2}\rho u^2 S b/4$ : 1.44 N m for the 6 m/s case and 3.24 N m for 9 m/s.

## 4.7 Test configuration and scenarios

Each profile is flown by both controllers in three conditions, Table 4. Conditions 2 and 3 add moment pulses on top of the wind, Table 5. Condition 1 has neither and is the baseline.

**Table 4.** Test matrix. Every case uses the same aircraft model and mission; PID and INDI differ only in the INDI parameter block. Wind speed is also given as a fraction of the 17 m/s cruise airspeed.

| Condition  | Wind [m/s] | Wind [km/h] | Fraction of cruise | Moment pulse |
|------------|------------|-------------|--------------------|--------------|
| 1 calm     | 0          | 0           | —                  | none         |
| 2 moderate | 6          | 21.6        | 35 %               | 1.44 N m     |
| 3 strong   | 9          | 32.4        | 53 %               | 3.24 N m     |

**Table 5.** Moment-pulse schedule, conditions 2 and 3. Each window starts a fixed time after an autopilot event, measured on the vehicle clock, so PID and INDI receive the pulses at the same point of the mission. Each window is a roll pulse, 4 s rest, then a pitch pulse: eight pulses per run.

| Window                                         | Starts                           | Pulses      |
|------------------------------------------------|----------------------------------|-------------|
| Take-off hover                                 | 4 s after take-off begins        | roll, pitch |
| Transition                                     | 3 s after transition starts      | roll, pitch |
| Fixed-wing                                     | 3 s after the fourth waypoint    | roll, pitch |
| Landing hover                                  | 5 s after landing descent starts | roll, pitch |
| Pulse: 1 s, body axis, magnitude from Table 4. |                                  |             |

The altitude controller is ArduPilot’s in both cases; the INDI build adds nothing to the vertical loop. Each log is split into hover, forward transition, wing-borne

flight, back transition and landing from the logged transition state, airspeed and lift-motor output, with the same rule for PID and INDI.

## 4.8 Metrics and analysis procedure

The pipeline reads each log, keeps the recorded units, and splits the mission into phases. ArduPilot logs body rates in  $^\circ/\text{s}$ ; an earlier analysis converted them from radians a second time and overstated every rate error by  $180/\pi$ . Each phase interval is filtered and transformed separately, so the jump between two windows cannot create false high-frequency energy. For  $N$  logged error samples  $e_k$ , the tracking error is

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N e_k^2}. \quad (17)$$

The stability measure is the RMS angular acceleration from the gyro over 6 Hz to 60 Hz. It uses the gyro, which both controllers share, rather than a signal that exists only inside INDI. Pseudo-control tracking is tested below 5 Hz on each phase interval: correlation tests direction and  $\text{RMS}(\dot{\omega}_f)/\text{RMS}(\nu)$  tests amplitude. Other metrics are attitude and rate RMSE, cross-track error, the share of commands above 0.95 of full scale, crab angle, and the fraction of time each axis runs INDI.

The comparison is paired: same mission, same wind, same pulses, only the controller changes. Once the two controllers respond differently their paths also differ, so later parts of a run are no longer a strictly controlled comparison. Differences smaller than the spread across repeated seeds are not claimed as results.

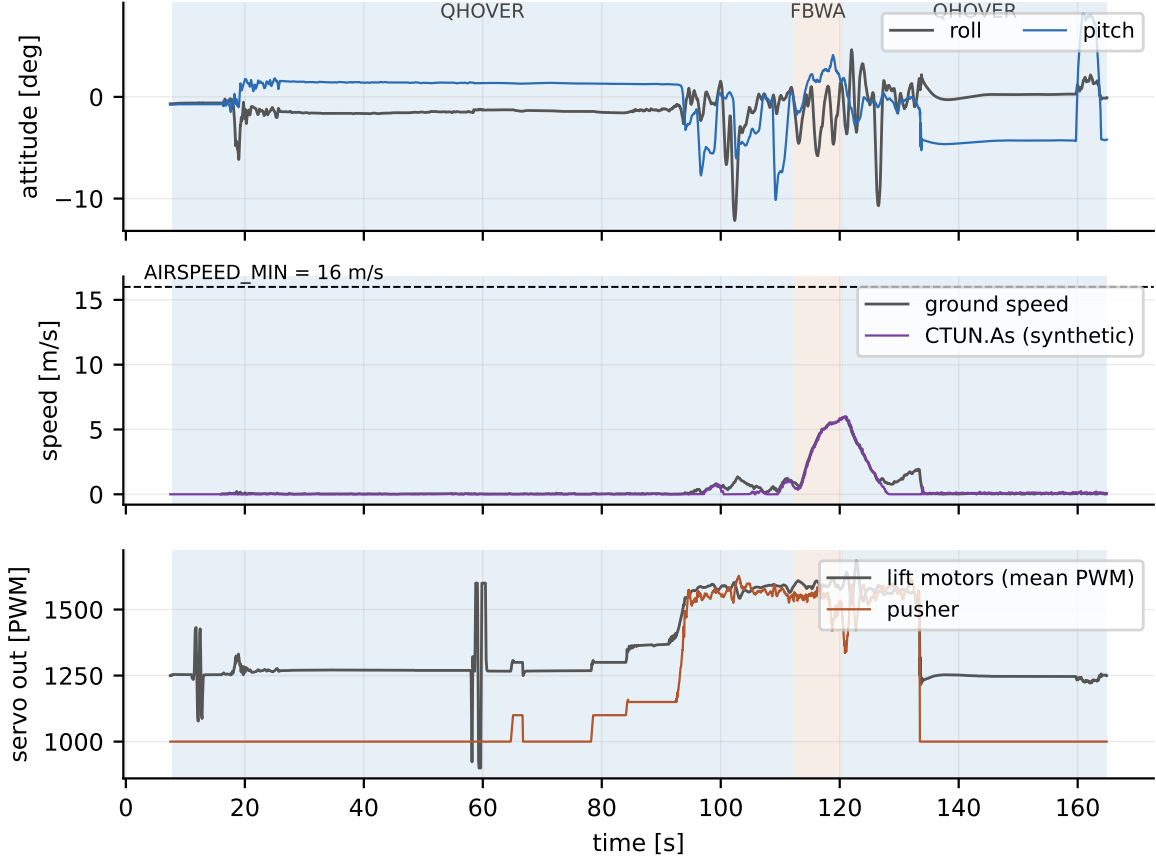
# 5 Results and Discussion

## 5.1 What the real flight log can and cannot establish

The one physical-flight record available is a 157.1 s stock-PID log. Mode segmentation gives hover mode from 7.8 s to 112.1 s, stabilised fixed-wing mode from 112.1 s to 120.5 s, and hover mode again to the end. Thus only 8.4 s, or 5 %, is fixed-wing operation. Ground speed in that fixed-wing interval averaged 3.7 m/s and reached 5.9 m/s; the logged airspeed reached only 6.0 m/s, well below the 16 m/s minimum-airspeed setting. The log contains no airspeed-sensor messages. The physical aircraft was therefore not demonstrably wing-borne, and the recorded

fixed-wing interval cannot validate the transition corridor or the wing model.

**F1** The real flight: 95 % hover, 8.4 s of FBWA below 6 m/s  
shaded blue = VTOL modes, orange = fixed wing



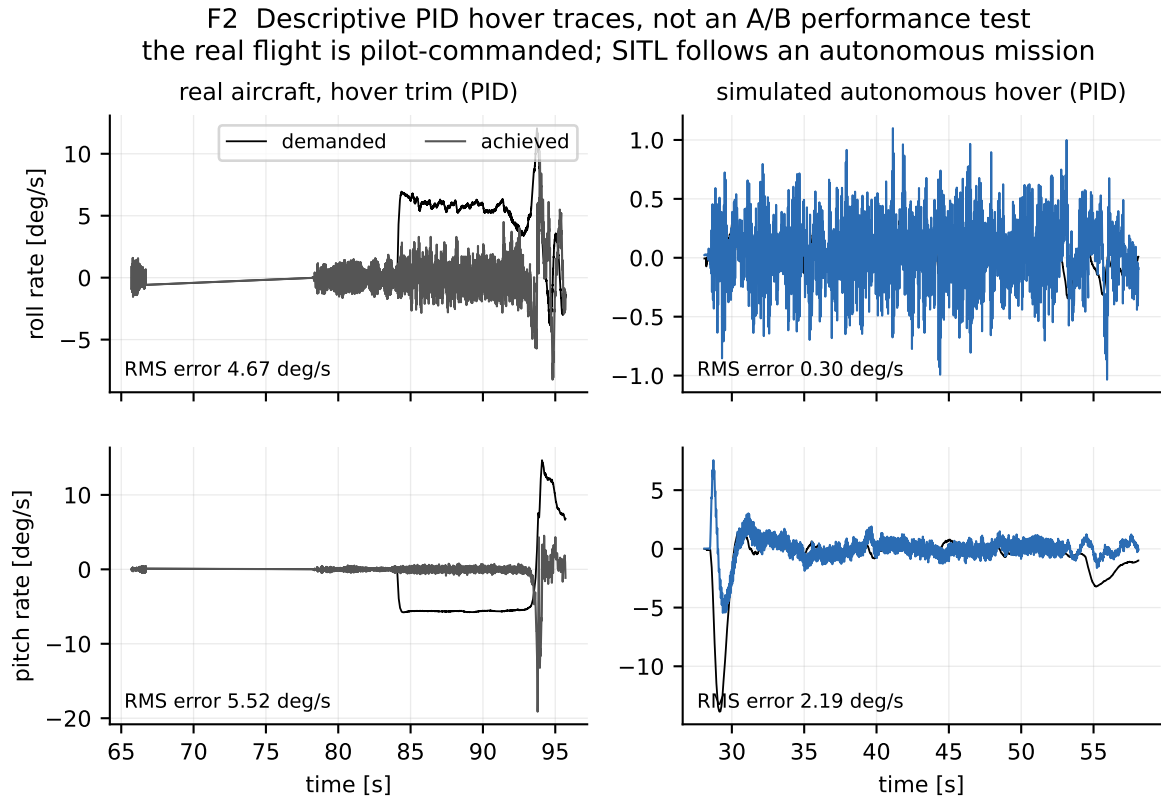
**Figure 4.** Purpose: establish the operating envelope of the only physical-flight dataset before using it for validation. The flight is predominantly in hover mode; the short fixed-wing interval remains below 6 m/s. This plot is the evidence for excluding real transition and cruise validation from the present study.

The log does provide three useful hardware facts. First, the learned hover throttle and the stored hover-throttle setting both equal 0.408, providing a configuration and hover-estimator consistency check. Second, it records the real scheduler rate (300 Hz), the 42 Hz gyro filter, and the vibration spectrum quantified in Table 8. Third, it shows the actuator and sensor logging needed for a future matched PID experiment. These are constraints on the simulation and controller design; they are not proof that Gazebo reproduces the real rigid-body or aerodynamic response.

Conversely, this dataset cannot support aerodynamic trim validation, transition tracking validation, control-effectiveness identification, gust-rejection performance,

or a hardware PID-versus-INDI comparison. Fixed-wing trim requires measured airspeed and a sustained segment long enough to average the phugoid; both are absent. The hand-flown rate content also lacks the independent, band-limited excitation needed to identify the inner-loop plant. The defensible result is consequently a constrained sim-to-real assessment: the simulation under-represents the differentiated-gyro noise substantially, while dynamic fidelity over the transition and cruise envelope remains unvalidated.

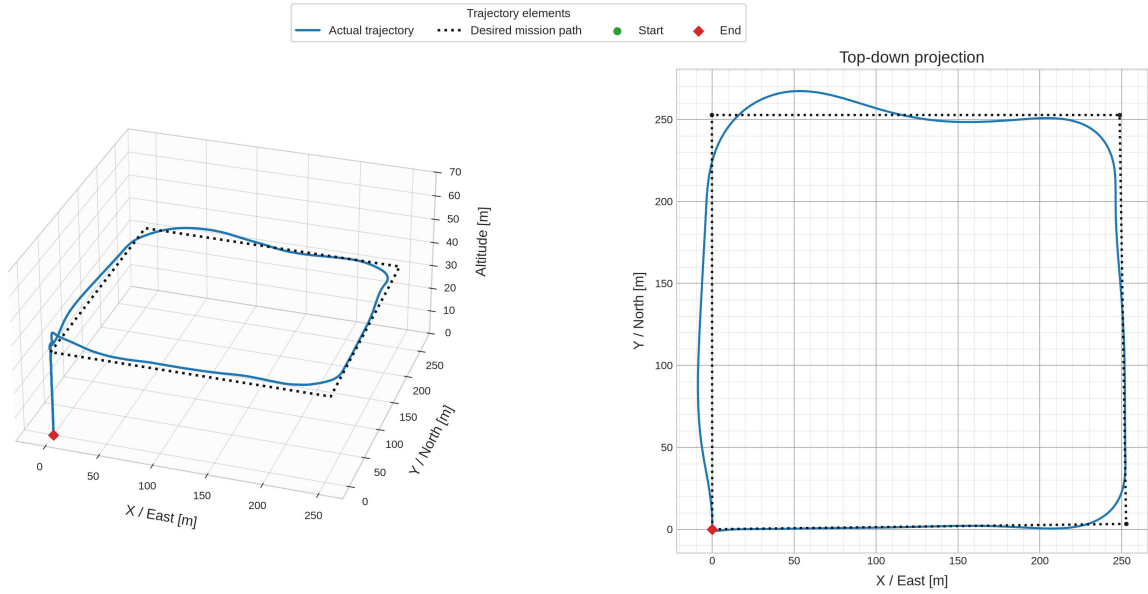
Figure 5 places representative real and simulated PID hover-rate traces beside one another to show why time-domain appearance alone is not a validation metric. The real window has roll and pitch RMS rate errors of  $4.67^\circ/\text{s}$  and  $5.52^\circ/\text{s}$ ; the autonomous calm-SITL window has  $0.30^\circ/\text{s}$  and  $2.19^\circ/\text{s}$ . The windows have different commands, noise, and pilot/autopilot excitation, so the smaller simulated values cannot be attributed to a more accurate model or better controller. Their purpose is descriptive: they show the data available and motivate the matched experiment still required.



**Figure 5.** Purpose: show representative PID rate tracking in the real and simulated hover records. The panels are deliberately labelled as an unmatched descriptive comparison; their RMS values must not be read as an A/B controller result.

## 5.2 Baseline simulated mission path

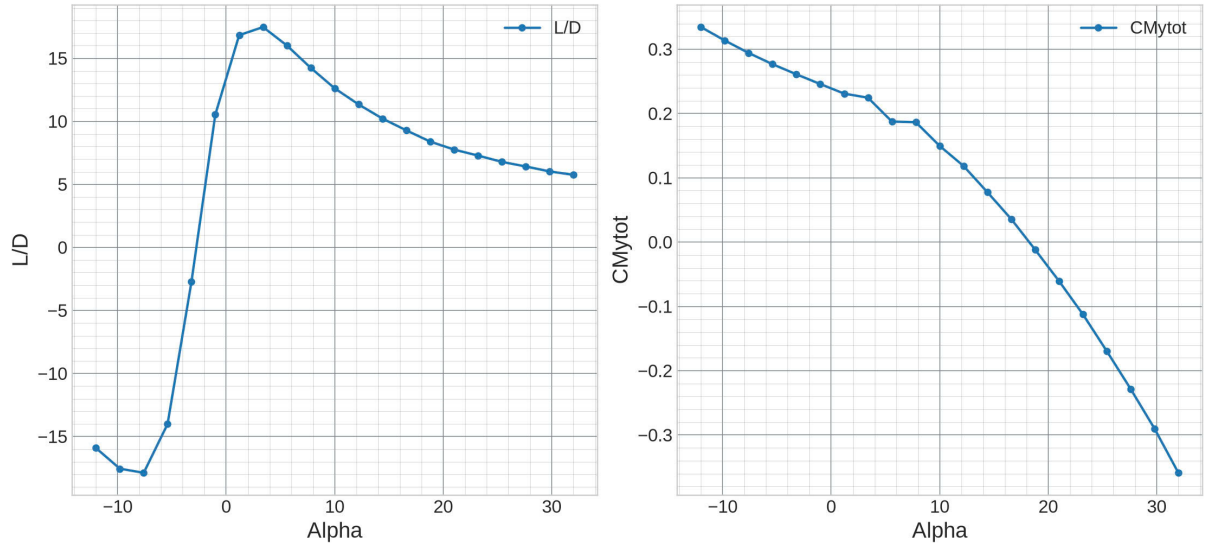
The calm and wind-enabled PID mission runs provide the geometric baseline for the rest of the study. Fig. 6 shows the commanded rectangular path against the simulated vehicle track under a 5 m/s wind and a 19 m/s cruise setting. The plot is useful because it shows that the simulator and mission scripts can complete the same end-to-end profile used later for the INDI comparison, including the transition into and out of forward flight. It does not, by itself, establish that the plant model is correct or that the real vehicle would follow the same path.



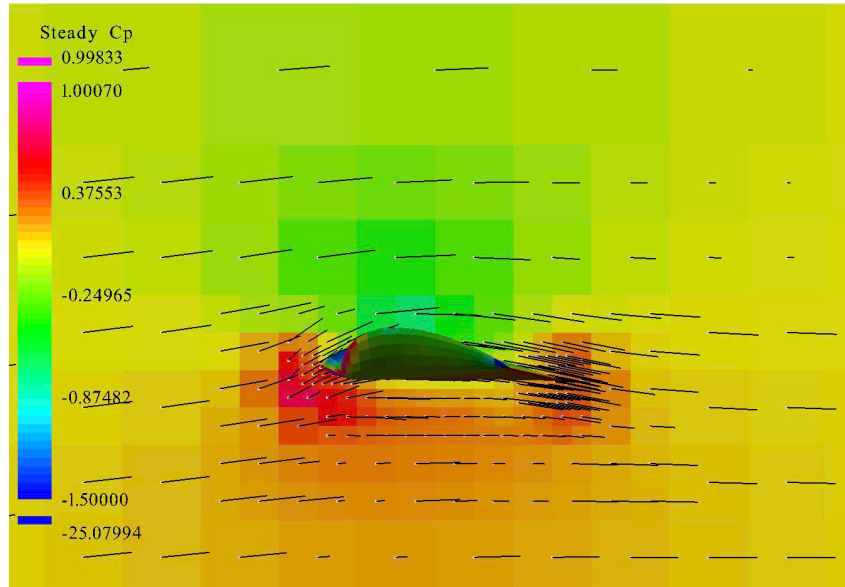
**Figure 6.** Simulated PID trajectory tracking versus the desired mission path, 3D view (left) and top-down projection (right), under a 5 m/s wind at 19 m/s cruise airspeed.

## 5.3 Aerodynamic model

Figure 7 shows the OpenVSP sweep of the full airframe and Fig. 8 the pressure field near the fuselage/wing junction. These are not flight results. They are the reference against which the Gazebo model is checked.



**Figure 7.** OpenVSP-derived aerodynamic performance of the full airframe: lift-to-drag ratio and total pitching-moment coefficient versus angle of attack  $\alpha$  (deg).



**Figure 8.** OpenVSP surface pressure coefficient and local flow-velocity vectors near the fuselage/wing junction, used to sanity-check the centre-of-pressure location.

**Model check.** Every lift-drag panel of the flown model was ranked by the maximum lift it can make,  $S C_{L,\max}$  with  $C_{L,\max} = C_{L\alpha} \alpha_{\text{stall}}$  (Table 6). The script is listed in Appendix C.

**Table 6.** Lifting surfaces of the flown Gazebo model, ranked by maximum lift. Share is of the wing and horizontal-tail total, 0.428 m<sup>2</sup>. Left and right panels are combined.

| Surface                  | $S$ [m <sup>2</sup> ] | $C_{L\alpha}$ [1/rad] | $\alpha_{\text{stall}}$ | $C_{L,\text{max}}$ | $S C_{L,\text{max}}$ [m <sup>2</sup> ] | Share |
|--------------------------|-----------------------|-----------------------|-------------------------|--------------------|----------------------------------------|-------|
| Main wing (centre panel) | 0.1832                | 3.70                  | 19.4°                   | 1.25               | 0.230                                  | 54 %  |
| Aileron sections (2)     | 0.0177                | 6.80                  | 36.6°                   | <b>4.35</b>        | 0.077                                  | 18 %  |
| Elevator                 | 0.0155                | 6.80                  | 36.6°                   | <b>4.35</b>        | 0.067                                  | 16 %  |
| Horizontal tail          | 0.0425                | 3.70                  | 19.4°                   | 1.25               | 0.053                                  | 12 %  |

Four problems follow from the table and the model file.

1. **The wing coefficients are not from OpenVSP.** The main-wing and aileron values ( $\alpha_0$ ,  $C_{L\alpha}$ ,  $C_{D\alpha}$ ,  $\alpha_{\text{stall}}$ ) are identical, to every digit, to ArduPilot’s example Zephyr flying-wing model. Only the areas and centres of pressure were changed, and the areas do match OpenVSP (0.201 against 0.204 m<sup>2</sup>). OpenVSP gives a lift slope of 5.98/rad and a zero-lift offset of 0.049 rad; the model uses 3.70 and 0.130 (Table 2).
2. **The aileron and elevator panels are unrealistic.** A lift coefficient of 4.35 at a stall angle of 36.6° is far beyond any plain airfoil (about 1.2–1.6 at 12°–16°). The elevator, a 0.015 m<sup>2</sup> strip, can lift more than the whole horizontal tail in front of it, and the two aileron sections add 33 % to the maximum lift of the wing.
3. **No panel carries a pitching moment** ( $C_{m\alpha} = 0$  everywhere). The OpenVSP pitching-moment curve is not in the model; pitch stability comes only from where the panels sit.
4. **The weight used earlier was wrong.** The earlier stall estimate used 27.5 N, which is the fuselage link alone. The model’s twelve links total 3.184 kg, a weight of 31.2 N.

With the correct weight, the 1-g stall speed  $V_s = \sqrt{2W/(\rho \sum S C_{L,\text{max}})}$  is 15.0 m/s for the centre wing panel alone, 13.0 m/s with the aileron sections, and 11.0 m/s with every horizontal panel. In the calm INDI run the aircraft held 46.9° of bank at 16.75 m/s, which is equivalent to 13.8 m/s in level flight, without stalling. That is below the centre-wing bound. The model can fly there only because of the extra lift of the aileron sections, whose  $C_{L,\text{max}}$  is not physical. The 14 m/s mini-

mum airspeed of Section 4.5 is therefore safe in this model but would not be on an aircraft whose outer wing stalls like a real airfoil.

Two further findings concern the control surfaces. The servo scaling for the rudders gives a travel of only about  $0.06^\circ$ , so the rudders do not move in any run; yaw in wing-borne flight comes from the fins' passive weathercock stability alone. The ailerons and elevator travel  $\pm 25^\circ$ .

**Propellers.** In the model each propeller is two lift-drag panels on a spinning link, so thrust grows with the square of rotor speed,  $T_i = k_{T_i}\Omega_i^2$  and  $Q_i = k_{Q_i}\Omega_i^2$ . The constants are set by the blade panels, not by the bench measurements of Fig. 3. Yaw moment comes from the difference in rotor drag torque, which is governed by  $k_{Q_i}$  and is the least certain part of the propulsion model; this is one reason yaw stays on the PID (Section 5.7).

$$T_i = k_{T_i}\Omega_i^2, \quad Q_i = k_{Q_i}\Omega_i^2 \quad (18)$$

These problems do not affect the PID/INDI comparison directly, because both controllers fly the same model. They do limit how far any simulated result transfers to the real aircraft (Section 5.12).

## 5.4 Identifying the aircraft that is being controlled

INDI needs one number per axis, the control effectiveness  $G$ , and it is the only model knowledge the law uses. Leaving it as a placeholder makes every downstream result arbitrary, so it was measured.

The mission flights cannot measure it, for the same reason the real flight cannot: an aircraft flying waypoints holds attitude, so the rate loop is excited only by what the navigation loop asks for. Fitting over that band returned  $G = 45.7 \text{ rad/s}^2$  on roll and nothing usable on pitch. A dedicated manoeuvre was therefore flown: a logarithmic stick sweep from 0.2 Hz to 6 Hz over 40 s per axis, in hover mode, with the rate loop on the *PID*. Identifying against INDI would measure a loop that already contains the number being sought.

Fitting  $\alpha(s)/u(s) = G/(1+s\tau)$  to the resulting frequency response gives Table 7.

**Table 7.** Control effectiveness and effector lag, identified in hover from a swept-stick manoeuvre with the rate loop on the PID.

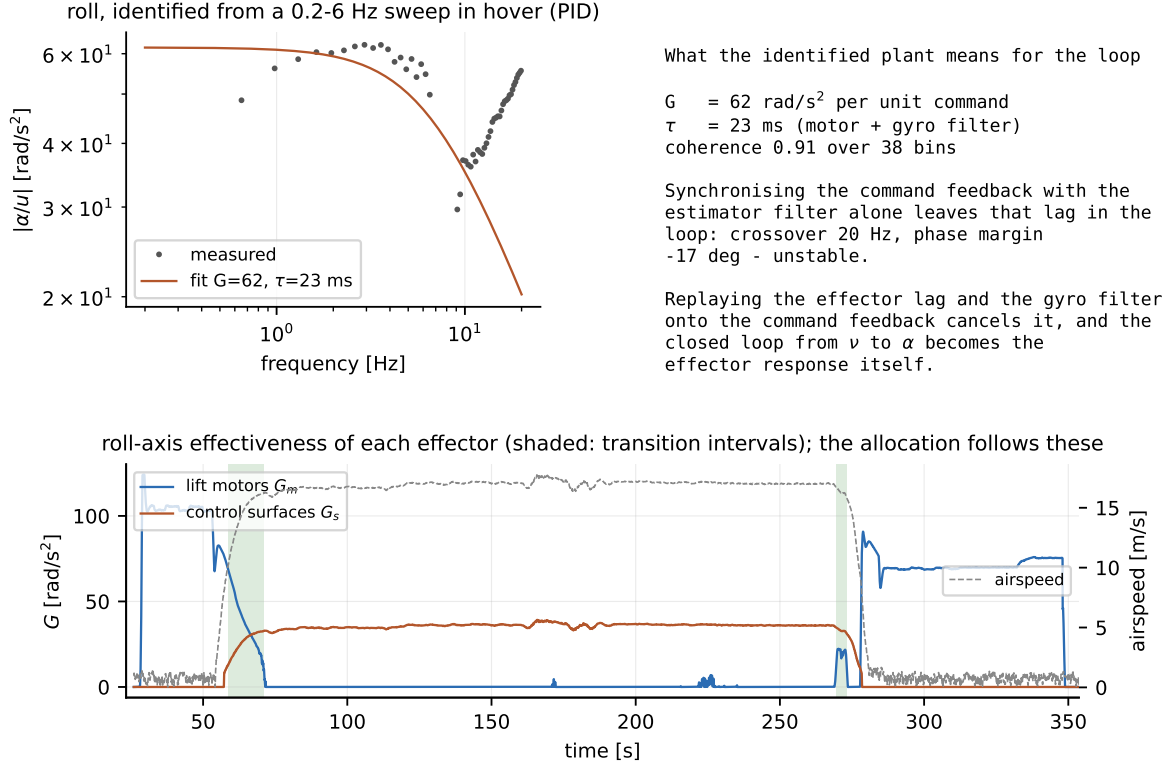
| Axis  | $G$ [rad/s <sup>2</sup> ] | $\tau$ [ms] | Coherence | Bins |
|-------|---------------------------|-------------|-----------|------|
| Roll  | 61.7                      | 23.0        | 0.91      | 38   |
| Pitch | 55.4                      | 23.7        | 0.89      | 37   |

Two independent checks support these numbers. The two axes agree on  $\tau$  to within 1 ms, which is what four shared motors should give. The pitch-to-roll effectiveness ratio is 0.90, against 0.79 predicted from the model’s own geometry and inertia,  $(x_{\text{arm}}/I_{yy}) / (y_{\text{arm}}/I_{xx})$ .

The identified  $\tau$  also disposes of an earlier misreading. A fit over the mission band alone had returned 88 ms; with excitation to 6 Hz the same aircraft returns 23 ms. A first-order model fitted to a band containing almost no high-frequency information will return whatever time constant makes the low-frequency magnitude work, and that value should not be used.

Figure 9 gives the identification result and its operational consequence in one plot. The upper-left panel shows the coherent command-to-angular-acceleration data and the fitted roll model,  $G_\phi = 61.7 \text{ rad/s}^2$  per unit normalised command with  $\tau = 23.0 \text{ ms}$ . The lower panel shows the scheduled motor and surface effectiveness in the calm INDI mission. Surface effectiveness rises with dynamic pressure while motor effectiveness falls as the lift motors unload. The purpose is to show how authority moves from the lift motors to the surfaces through the transition.

#### F4 Control effectiveness: identified, then scheduled



**Figure 9.** Purpose: show where the simulated-aircraft inversion gain comes from and how authority is handed from lift motors to aerodynamic surfaces. The fit is from the dedicated PID sweep; the time history is from the calm INDI mission. These values are not hardware identification results.

### 5.5 The dominant sim-to-real risk: vibration in the angular-acceleration estimate

INDI does not measure angular acceleration. It differentiates the gyro. Differentiation multiplies a noise component at frequency  $f$  by  $2\pi f$ , so gyro noise far above the control bandwidth still arrives at the inversion as angular acceleration. The quantity that matters is therefore not the gyro amplitude but the angular-acceleration noise inside the band the estimator passes.

This was measured on the real aircraft from the batch-sampled gyro log, recorded at 1916 Hz through 94 s of hover. The gyro power spectrum was multiplied by  $(2\pi f)^2$  and integrated over 25 Hz to 90 Hz: the lower edge sits above the rate-loop bandwidth so that rigid-body motion is excluded, and the upper edge is below the 150 Hz Nyquist frequency of the 300 Hz control loop. The same treatment was

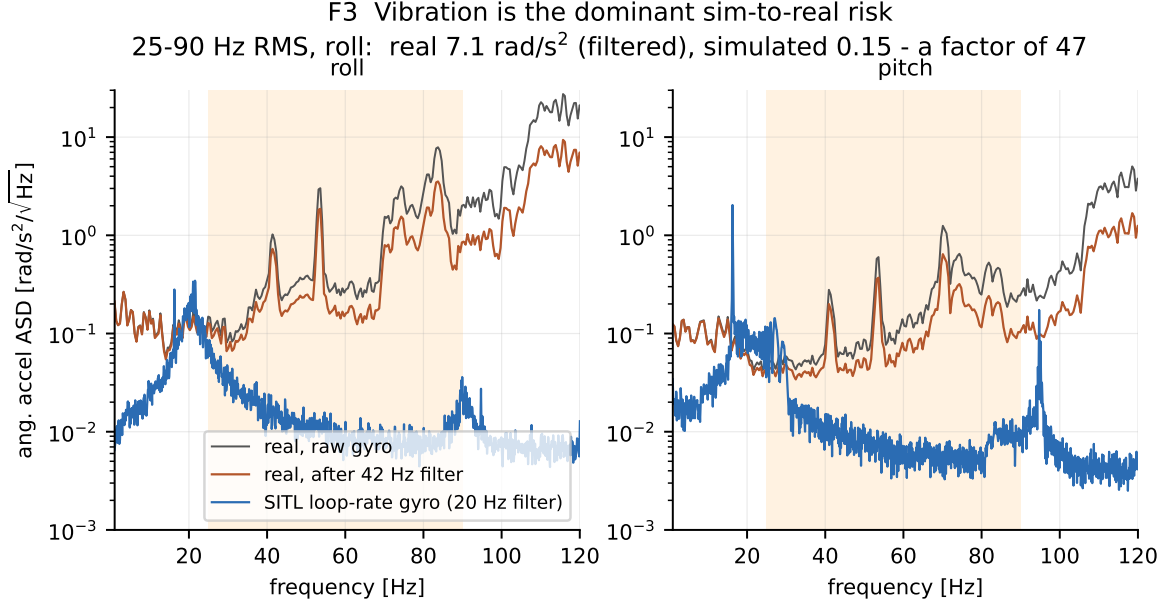
applied to the simulated aircraft in hover.

**Table 8.** Angular-acceleration noise in the estimator band, 25 Hz to 90 Hz, hover. The real-aircraft column applies the 42 Hz gyro low-pass filter recorded in the physical log; the calm-PID SITL result uses its logged 20 Hz filter.

| Axis  | Real, raw<br>[rad/s <sup>2</sup> ] | Real, filtered<br>[rad/s <sup>2</sup> ] | Simulated<br>[rad/s <sup>2</sup> ] | Ratio       |
|-------|------------------------------------|-----------------------------------------|------------------------------------|-------------|
| Roll  | 15.1                               | <b>7.1</b>                              | 0.15                               | <b>46.6</b> |
| Pitch | 2.6                                | 1.3                                     | 0.21                               | 6.2         |
| Yaw   | 2.0                                | 1.1                                     | 0.09                               | 11.7        |

The roll axis is dominated by a spectral line near 83 Hz; pitch near 71 Hz and yaw near 53 Hz. Accelerometer vibration is unremarkable on the same flight (logged vibration levels of 1.25, 1.12 and 1.58 m/s<sup>2</sup> with no clipping), which is why this had not been noticed: the standard vibration check looks at the accelerometers, and the problem is on the gyro, in a band that check does not examine.

The direct comparison is deliberately limited to the estimator input; it is not an end-to-end airframe-fidelity score. Even after the physical aircraft’s wider gyro filter is applied, the real roll-axis acceleration noise is 7.13 rad/s<sup>2</sup> RMS, compared with 0.153 rad/s<sup>2</sup> RMS in calm PID SITL: a factor of 46.6. Figure 10 shows the spectra from which those integrals are computed. This mismatch is large enough that a rate-loop cutoff or increment limit accepted in this simulator cannot be transferred to the aircraft without hardware vibration measurements.



**Figure 10.** Purpose: quantify the differentiated-gyro environment that enters the INDI inversion. The shaded 25 Hz to 90 Hz band is integrated in Table 8. The result validates only the sensor-noise mismatch; it does not validate rigid-body or aerodynamic dynamics.

Three consequences follow. First, any rate-loop tuning obtained in this simulator is optimistic because the estimator cutoff and increment limit are chosen against a noise floor roughly 47 times too low on the worst axis. Second, the choice of estimator cutoff on hardware is not a tuning preference but a measurement problem: it has to be made from a measured hardware vibration spectrum, trading estimator lag against the noise this table quantifies. Third, the mismatch is a property of the airframe and its mounting, not a property of the control law, so the most effective response is mechanical rather than algorithmic.

## 5.6 Verifying that the simulated wind is a wind

A crosswind study is only meaningful if the aircraft feels the wind rather than merely being told about it. These are separable failures: the autopilot’s airspeed arrives from the simulator over the JSON interface, so it can be entirely correct while the aerodynamic model ignores the wind.

Two defects were found and fixed before any disturbed case was flown. The Gazebo wind was enabled on a model that the world does not spawn, so no link was wind-enabled at all. Once enabled, Gazebo’s wind plugin applied its default force approximation,  $F = m k (v_{\text{wind}} - v_{\text{link}})$  with  $k = 1$ : a velocity-proportional body

drag of 2.8 N per m/s on the 2.8 kg fuselage link, roughly 48 N at cruise against a weight of 31 N. The forward transition simply stopped happening – airspeed stalled at 4.6 m/s and the aircraft flew the whole mission as a multicopter. That approximation exists for objects with no aerodynamics of their own; this aircraft carries a lift-drag model on every surface, so the force approximation was disabled and the wind left to act through the air-relative velocity those surfaces already compute.

The check that the fix worked is the crab angle, because an aircraft whose wings feel a crosswind must point away from its ground track to hold it. In the current campaign, the 6 m/s crosswind case has mean airspeed 16.7 m/s, ground speed 15.5 m/s, and a mean crab angle of 18.8°; the 9 m/s case has a 32.3° mean crab angle. The wind therefore enters the aerodynamic model rather than only the navigation state. These are steady-wind checks, not gust-rejection results: no repeatable gust waveform was injected or measured.

## 5.7 Yaw authority

Yaw is excluded from INDI in this work, and the reason is measured rather than argued. During the identification sweep – with yaw on the stock rate PID throughout – the yaw command sat on its  $\pm 1$  limit for 48 % of the manoeuvre, and it reaches 22 % to 26 % during hover and transition in the 9 m/s crosswind case.

An axis that spends a quarter to a half of its time saturated has no authority margin left for an inversion to allocate, and its effectiveness cannot be identified from data in which the command is against a stop for much of the record. Yaw moment on this airframe comes from rotor drag torque rather than from differential thrust, so it is also the axis whose effectiveness the propulsion model describes least well (Section 5.3). Setting the yaw effectiveness to zero routes the axis to the stock PID through the same per-axis fallback that handles an unspooled motor, and the axis is reported as out of scope rather than quietly included. This is an airframe and mixer finding, not a control-law one.

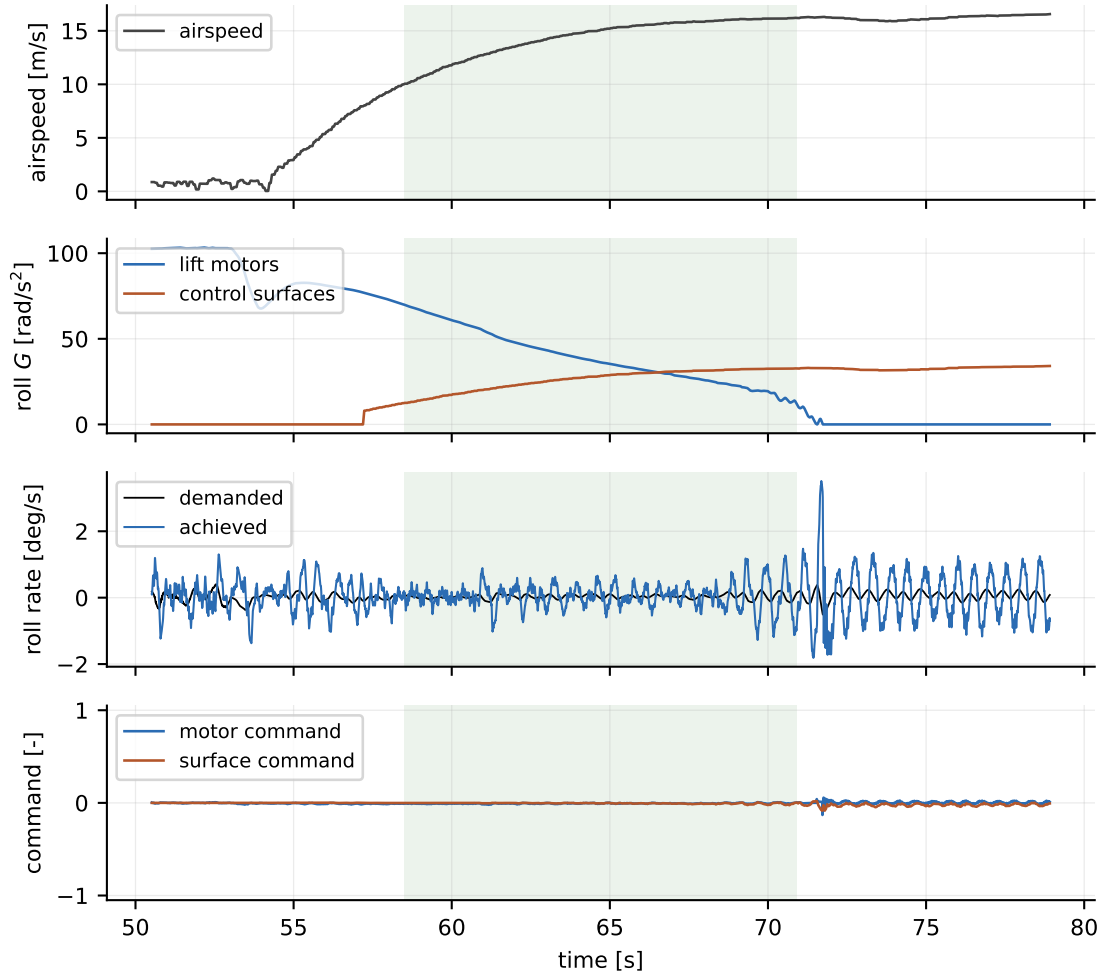
## 5.8 Fixed-wing INDI during transition and forward flight

All six planned SITL missions completed: PID and INDI in calm, 6 m/s, and 9 m/s crosswinds. In the calm INDI run, measured-state segmentation gives 100.9 s

of hover, 16.1 s of transition, and 202.6 s of forward flight. Lift-motor roll and pitch INDI were active on every logged hover and transition sample; surface roll and pitch INDI were active on every transition and forward-flight sample. In forward flight motor-side activity fell to 4%, consistent with the lift motors unloading. Yaw remained on the stock PID path, as intended.

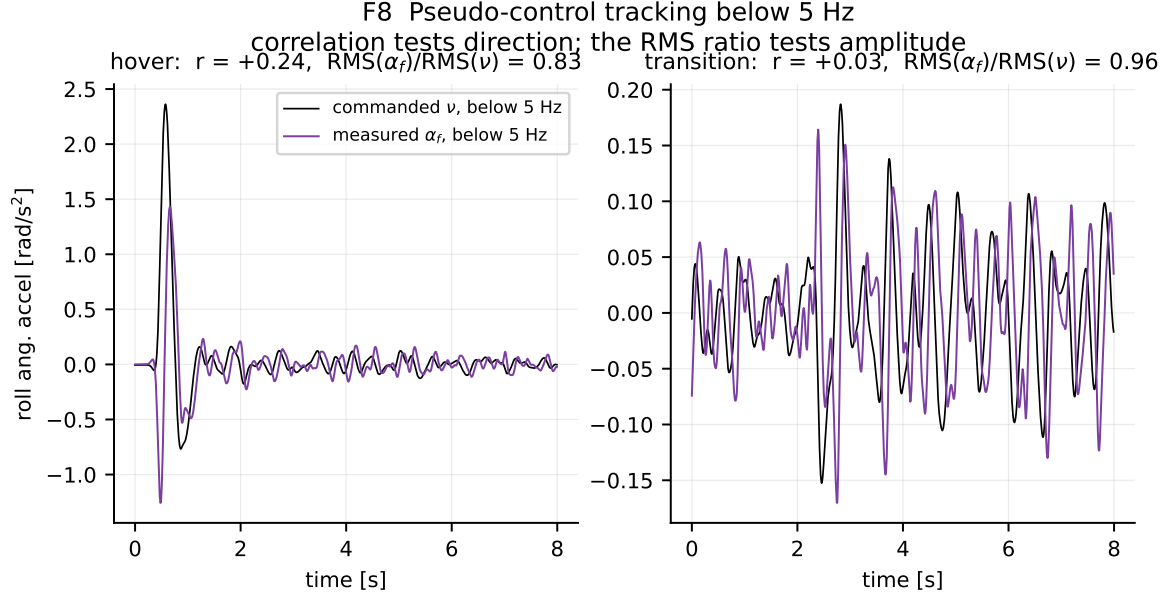
Figure 11 shows the first calm forward transition. Its purpose is to make four claims auditable on a common time axis: airspeed crosses into the wing-borne regime, surface effectiveness grows, motor effectiveness decreases, and the commands and measured roll-rate response remain bounded. This demonstrates that the handover executes; it does not by itself establish that the requested pseudo-control was accurately realised.

F6 The forward transition (shaded): effectiveness hands over from motors to surfaces with the resulting rate response and commands shown below



**Figure 11.** Purpose: expose the calm forward transition as a time history. The shaded interval shows the simultaneous motor/surface regime; the lower panels report the resulting roll-rate response and normalised commands.

The internal pseudo-control test in Fig. 12 supplies the missing qualification. In calm hover, the filtered roll acceleration has the correct amplitude ratio, 0.83, but only weak correlation with the requested acceleration,  $r = 0.24$ . During transition the amplitude ratio remains 0.96 while correlation falls to  $r = 0.03$ . Thus the measured acceleration has approximately the requested RMS magnitude but is not demonstrably following its time variation. The 6 m/s run is similar in transition ( $r = 0.15$  roll and 0.08 pitch). At 9 m/s, pitch becomes anti-correlated ( $r = -0.19$ ). Mission completion is therefore integration evidence, while acceleration tracking through the corridor remains weak.

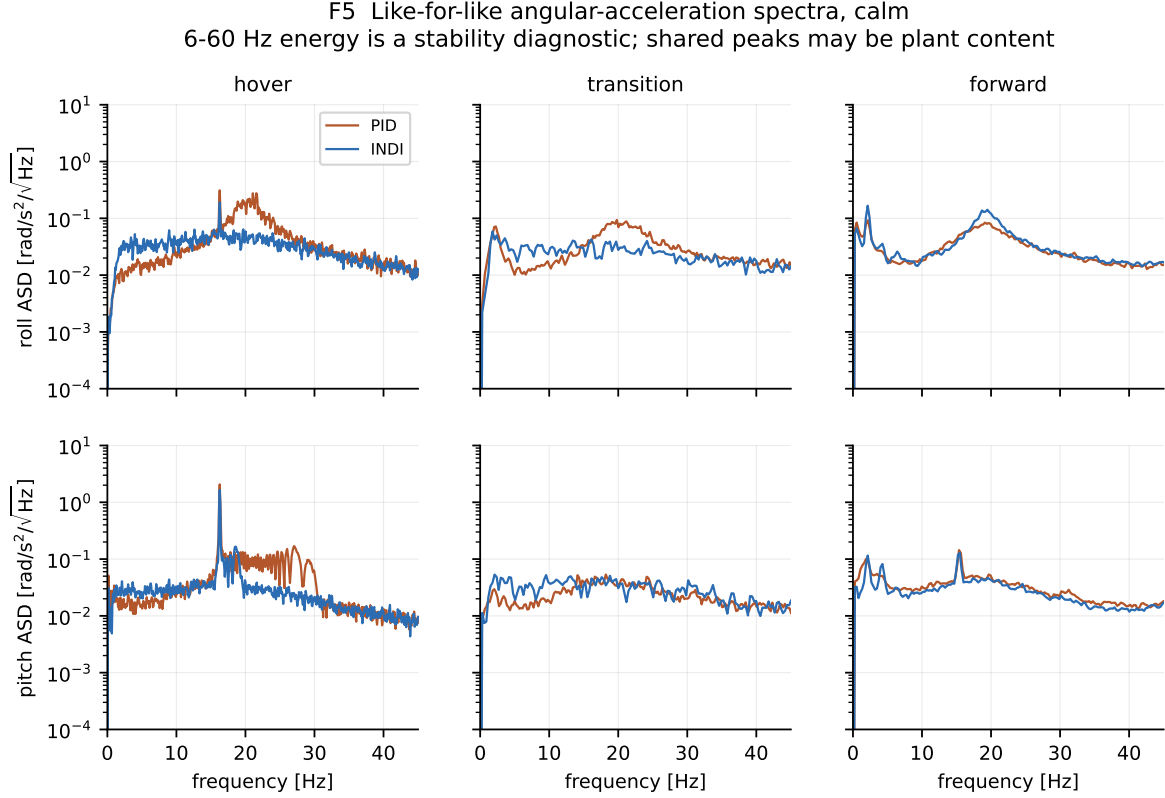


**Figure 12.** Purpose: test the INDI law on its own terms by comparing requested and measured roll angular acceleration below 5 Hz. Statistics use every contiguous phase window; the traces show one representative window. Weak transition correlation prevents a strong tracking claim.

The earlier crosswind runs verified allocation under persistent aerodynamic load but did not isolate transient rejection. The rework campaign in Section 5.10 adds repeated, specified roll/pitch moment pulses to each crosswind case. These are applied torques, not measured atmospheric gusts; their response must not be described as gust rejection.

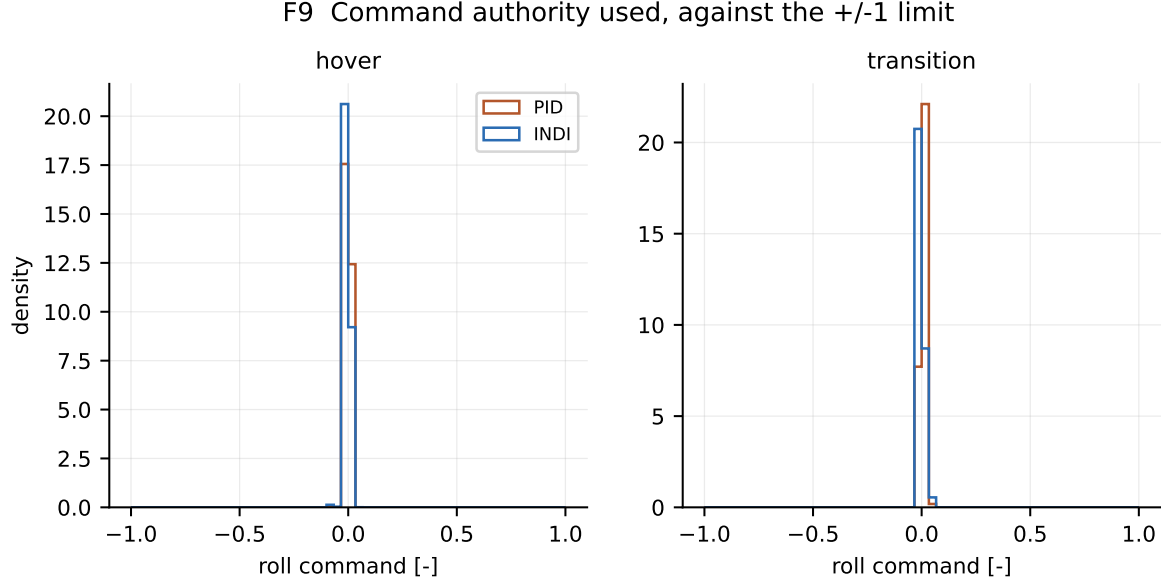
## 5.9 Stability and control authority

Figure 13 compares PID and INDI using the same gyro-derived angular-acceleration spectrum in the calm missions. This avoids the earlier error of diagnosing a controller from an internal signal that did not exist under PID. In hover, INDI reduces the 6 Hz to 60 Hz RMS from 0.459 to 0.229 rad/s<sup>2</sup> on roll and from 0.852 to 0.644 rad/s<sup>2</sup> on pitch. In transition, roll falls from 0.248 to 0.164 rad/s<sup>2</sup>, while pitch is nearly unchanged (0.190 PID, 0.200 INDI). In forward flight, INDI roll energy is higher (0.310 versus 0.243 rad/s<sup>2</sup>). The 16.3 Hz hover line appears under both controllers, so its presence alone cannot be assigned to INDI. The figure supports stability in the sense of bounded spectra and no unique narrow INDI line in calm flight; it does not identify the source of the shared airframe resonance.



**Figure 13.** Purpose: compare high-frequency angular-acceleration content under PID and INDI using a signal logged under both controllers. The paired calm missions are shown by flight phase. Shared peaks are treated as plant/model content rather than automatically attributed to INDI.

Figure 14 checks whether these results are explained by roll-command saturation. They are not: no roll sample exceeds 0.95 in either calm mission. The 95th-percentile absolute demand is 0.0167 for PID and 0.0075 for INDI in hover, and 0.0227 versus 0.0210 in transition. These are pre-mixer axis demands, so they do not measure motor energy or surface work, but they show that the roll findings are not produced by the command rail.

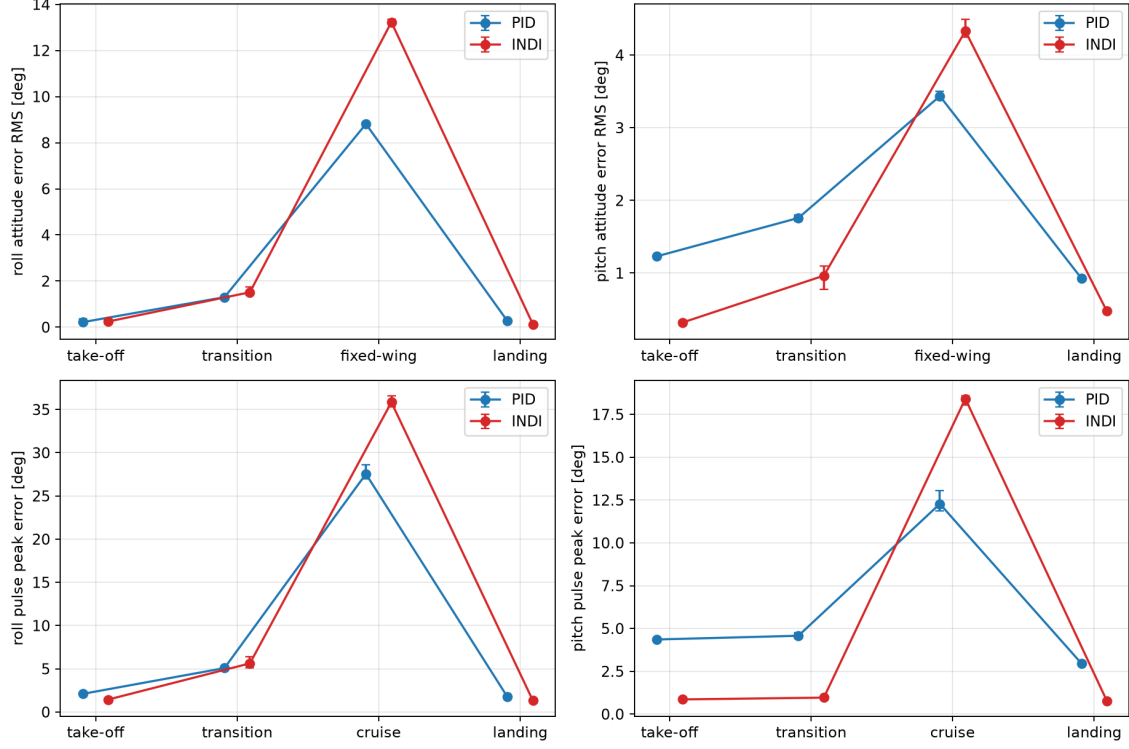


**Figure 14.** Purpose: test whether the calm roll results are authority-limited. Neither controller approaches the normalised  $\pm 1$  command limit in hover or transition.

## 5.10 PID and INDI comparison

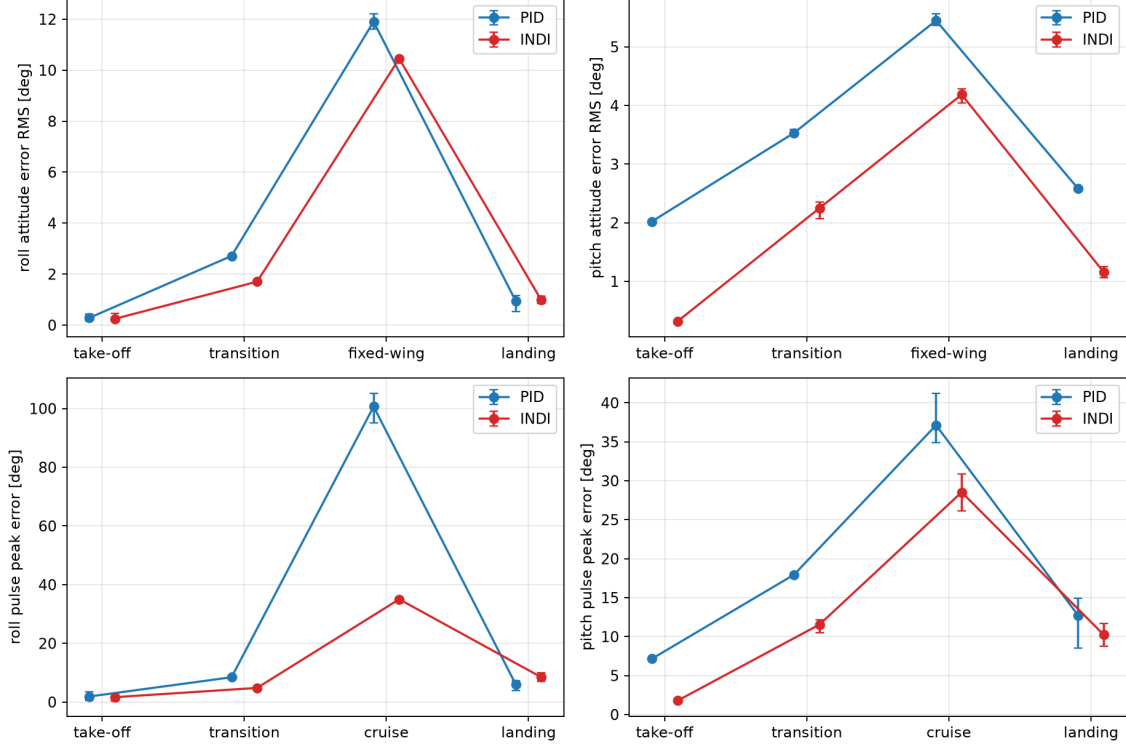
The rework campaign reran the same square mission with PID and INDI in 6 m/s and 9 m/s wind, three paired seeds per speed. Both controllers used the same binary and plan; only the INDI parameter block differed. All twelve noisy-sensor missions landed and passed the transition and commanded-pulse checks. Gazebo retained its 1 ms physics step while running headless without a wall-clock real-time cap. Seeded wind and pulse commands do not guarantee bitwise trajectory identity because the SITL–Gazebo connection is not fully lockstep. The range across three seeds is shown, not a confidence interval. Of 96 scheduled torque pulses, 92 can be individually resolved in the lower-rate servo-output log; the four missing marker transitions are excluded from pulse aggregates.

Figure 15 shows attitude error by phase and peak error during the scheduled moment pulses. At 6 m/s, take-off pitch RMS is  $1.23^\circ$  with PID and  $0.32^\circ$  with INDI; transition pitch is  $1.76^\circ$  and  $0.96^\circ$ , respectively. Fixed-wing roll RMS is worse with INDI ( $13.23^\circ$  versus  $8.81^\circ$ ). The fixed-wing pitch-pulse peak likewise grows from  $12.28^\circ$  to  $18.42^\circ$ ; the controller is not uniformly better.



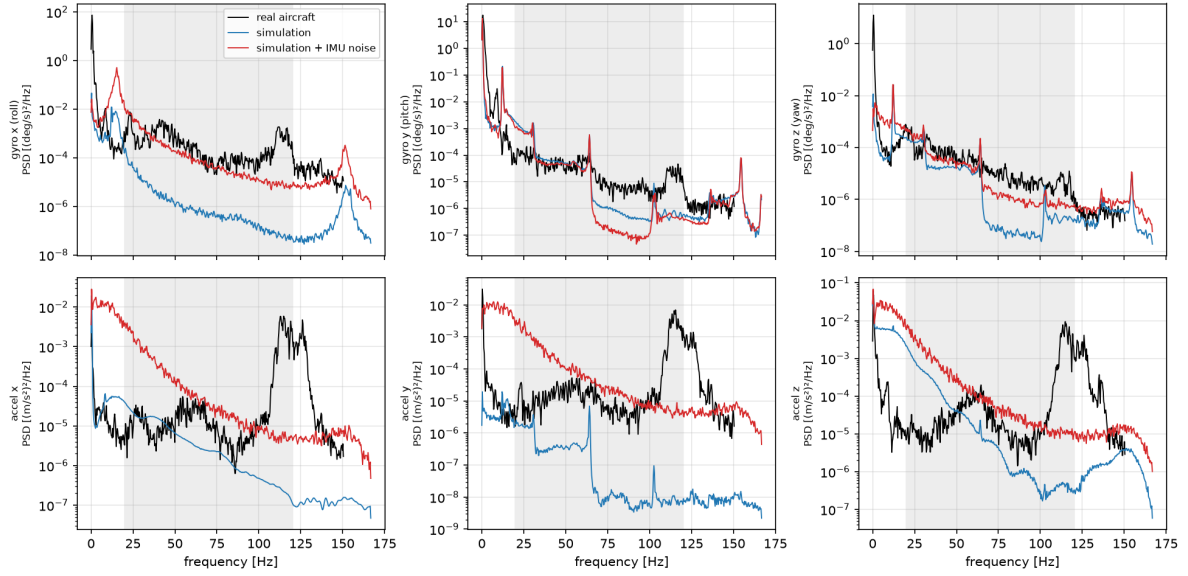
**Figure 15.** Paired 6 m/s wind experiment: roll and pitch tracking-error RMS by flight phase (upper row), and baseline-subtracted peak error in the five-second window after each scheduled one-second moment pulse (lower row). Points are means of detected pulses, normally over three seeds; whiskers show their minimum and maximum. A few take-off roll markers were not resolved by the lower-rate servo log, so their sample sizes are smaller (one or two; see the per-pulse CSV). Pulse windows are named by their trigger, which can differ from the actual flight phase at injection.

At 9 m/s, take-off pitch RMS is  $2.02^\circ$  for PID and  $0.32^\circ$  for INDI; transition roll RMS is  $2.71^\circ$  and  $1.70^\circ$ . The cruise roll-pulse peak averages  $100.67^\circ$  for PID and  $25.72^\circ$  for INDI. This is not evidence for an unassisted fixed-wing improvement: the lift motors assisted in the fixed-wing phase for both controllers, averaging 33.4 % of its duration for PID and 10.6 % for INDI. Differences in trajectory, transition timing, and assist status limit causal attribution to the rate law alone.



**Figure 16.** Paired 9 m/s wind experiment with the same definitions as Fig. 15. Three seed-paired, full-physics missions were completed per controller; one take-off roll marker was missed in INDI and the cruise roll-pulse result includes assisted flight, not a purely wing-borne test.

The added Gazebo IMU noise was calibrated separately against a manual airborne hover interval from the real log. In an independent calm validation flight, post-filter 20 Hz to 120 Hz roll-gyro RMS was  $0.251^\circ/\text{s}$  in simulation versus  $0.254^\circ/\text{s}$  on the aircraft; the pitch gyro remained  $0.105^\circ/\text{s}$  versus  $0.060^\circ/\text{s}$  (74 % high). The acceleration-axis RMS values for simulation versus real were 0.177 versus 0.175 (x), 0.166 versus 0.169 (y), and 0.240 versus 0.205 (z), all in  $\text{m/s}^2$ . The gyro filter, loop rate, harmonic notch and tonal rotor vibrations still differ from the real aircraft; matching integrated noise power is not hardware validation.



**Figure 17.** Primary logged IMU spectra in manual real hover, a noise-free simulated square-mission landing hover and an independently flown simulated landing hover with calibrated sensor noise. Shading indicates the 20 Hz to 120 Hz band integrated for the six-axis RMS comparison. Unlike real motor vibration, the injected sensor noise is Gaussian; spectral shape and pitch-gyro power remain mismatched.

## 5.11 Discussion of findings

The real and simulated evidence answer different questions. The physical log is most valuable as a measurement of the hardware environment: it records the gyro-noise floor, filter configuration, scheduler rate, and the fact that the available flight never reached a valid fixed-wing condition. The SITL campaign is most valuable as an integration and handover test: the activity flags show which effector set runs INDI in each phase, and the crab-angle result confirms that the simulated aircraft feels the specified crosswind aerodynamically.

Taken together, the results expose the central sim-to-real risk rather than hiding it. Before noise injection, the original simulator produced roughly 47 times less roll-axis acceleration noise than the physical aircraft in the estimator band. The rerun matches integrated roll-gyro hover noise more closely, but not the real tonal spectrum, pitch-gyro noise, filter chain or airframe plant. The real flight provides no wing-borne data with which to validate the aerodynamic model. The seed-paired moment-pulse results show condition-dependent gains and costs, not a general guarantee of robust control.

The practical conclusion is staged. The present results support further SITL

tuning and a better hardware experiment; they do not support a claim of real-flight INDI performance, superior atmospheric gust rejection, or a general PID-versus-INDI advantage. Filter and sensor matching, plant fidelity and a safe real-flight test remain necessary before transferring the controller to the vehicle.

## 5.12 Limitations of the study

In the project The real dataset contains a manually flown PID record, no measured airspeed, and only 8.4s in forward flight; it cannot validate the wing, transition, or rate-loop plant. The Gazebo propeller model is valid only over its measured RPM range, and the simulated gyro environment is significantly cleaner than physical model, where noise from the motor and resonance of the frame have a significant role. The Yaw authority problem caused by the twisting of the frame is unmodeled in the simulation. Finally, the weak transition pseudo-control tracking and the failed 9 m/s INDI validation show that the present gains are not a robust final tuning. These limitations bound every performance claim in this thesis.

### 5.12.1 Problems encountered during implementation

Beyond the modelling limitations above, several problems were found while getting the controller working. They are reported here because most of them were invisible in the flight itself – the aircraft completed its mission in every case – and because the way each was found says something about what a simulation can and cannot be trusted to reveal.

**The parameter file was never being read.** The SITL defaults file was a five-column QGroundControl export. ArduPilot’s parser takes the first whitespace-separated token as the parameter name and the second as its value, so every line asked to set a parameter named after the export’s index column. Nothing in the file was applied, and each run instead booted from whatever the previous session had left in the autopilot’s stored memory. This is the real explanation for a symptom previously recorded as a single parameter refusing to load from defaults: no parameter was loading, and the ones that appeared correct were leftovers. It also means the configuration of any earlier result is not reconstructible. The file is now two-column, and the run harness gives each case a fresh stored memory so that the defaults file is the only source of parameters.

**Synchronisation was incomplete, and that was the limit cycle.** The first implementation filtered the acceleration estimate but added the increment to an unfiltered command, which is the defect Section 3.6 describes. Correcting it to filter the command with the same estimator filter was necessary but not sufficient: the rate loop still limit-cycled. Two further lags sit between commanding an effector and seeing the moment, and neither was on the command path – the effector’s own response, and the IMU gyro filter. Section 4.3 gives the analysis; the practical consequence is that with only the estimator filter replayed, the identified plant predicts a phase margin of  $-34^\circ$ , and the loop oscillated within 10 % of the predicted frequency. Replaying all three stages removed it.

**Wind that pushed the aircraft instead of blowing past it.** Gazebo’s wind plugin approximates the force on a wind-enabled link as  $F = m k (v_{\text{wind}} - v_{\text{link}})$  with  $k = 1$  by default, which on the 2.8 kg fuselage link is a velocity-proportional drag of roughly 48 N at cruise against a weight of 31 N. The forward transition stopped happening entirely. That approximation is intended for objects with no aerodynamic model of their own; this aircraft has a lift-and-drag model on every surface, so the force approximation was disabled and the wind left to act through the air-relative velocity those surfaces already compute (Section 5.6).

**An oscillation that was not the controller’s.** A line near 16 Hz in the hover angular-acceleration estimate was read as an INDI limit cycle through several iterations. Two things eventually contradicted that reading: the frequency did not move when the inversion gain was changed by 35 %, and the same line is present in the PID baseline when the comparison is made on the raw gyro rather than on the controller’s internal estimate – which only exists while INDI is running, and so had no baseline to be compared against. Measured properly, INDI has *less* energy in that band than the PID. The line is a property of the simulated airframe whose source has not been identified. The lesson is specific and worth stating: a diagnostic computed only inside the controller under test cannot distinguish the controller from the plant.

**Integration friction.** Several smaller difficulties came from working inside a mature codebase. The INDI controller is built on the attitude controller that ArduPilot already uses for this vehicle class, so only the rate loop is overridden and the angle loop, throttle handling and frame transforms stay stock. The mixer does not store the roll and pitch it actually applied after saturation, so the controller

recovers them each cycle from the individual motor outputs. The control loop in SITL does not run at its nominal rate: the simulator steps in whole milliseconds, so a 400 Hz loop actually runs every 3 ms. The derivative and every filter corner therefore use the measured loop period, not the nominal one.

## 6 Conclusion and Future Work

This thesis develops an incremental rate-control path for a quadplane, from the control-theory derivation to an INDI rate loop on the lift motors and control surfaces in ArduPilot, handed over between them by ArduPilot’s own transition logic. It explains why cascaded PID remains a practical baseline; why a full non-linear inverse is unsuitable when the aerodynamic model is uncertain; how the incremental law follows from both Taylor-series and sensor-based derivations; and why command, actuator, and measurement delays must be matched for the inversion to remain valid. The controller executes through hover, transition, and forward flight in Gazebo/SITL, with activity logging that separates genuine INDI ownership from PID fallback.

The real flight record constrains, rather than validates, the simulation. It confirms the hardware sensor environment and reveals a filtered roll-axis acceleration-noise level 47 times the calm-SITL value, but it contains neither measured airspeed nor enough fixed-wing operation to validate transition dynamics. Within SITL, the handover engages as designed and the 6 m/s case improves transition attitude RMSE, but the 9 m/s case produces  $2.45^\circ$  roll-attitude RMS error and  $11.07^\circ/\text{s}$  roll-rate RMS error. The results therefore demonstrate implementation and expose a robustness limit; they do not demonstrate real-aircraft INDI performance or a general PID-versus-INDI advantage. The immediate next step is a matched, instrumented PID validation flight and its replay in SITL, followed by repeated transient-disturbance experiments only after the model and sensor discrepancy have been addressed.

### 6.1 Future work

Most notably, the eventual goal of this line of work is to fly the INDI controller on the real vehicle, not just in simulation. That step is deliberately placed last and is gated on everything above: it should only be attempted once (i) the simulation

has been shown to reproduce real PID flight behaviour to an accepted tolerance, especially in yaw (Section 4.5), (ii) the corrected synchronization fix of Eq. (15) has been re-validated in SITL, and (iii) the standard hardware-in-the-loop progression is followed: bench and props-off checks, tethered hover, an immediate remote-controlled fallback switch back to PID, and only then a cautious, low-altitude, progressively-expanding real flight-test campaign. Further work beyond the first real flight includes online or scheduled effectiveness identification for the hover and fixed-wing gains alike, especially for yaw, which remains on the stock coordinated-turn law in both regimes (Section 4.4); tracking-accuracy validation of the fixed-wing extension against the metrics of Section 4.8, beyond the engagement result already obtained (Section 5.8); constrained control allocation for the overactuated transition and cruise mixer; and a genuine outer-loop (vertical/position) INDI law to replace the current bounded PID throttle correction, following the cascaded hover/fixed-wing structure demonstrated for a comparable vehicle by Zhou et al. [7].

## References

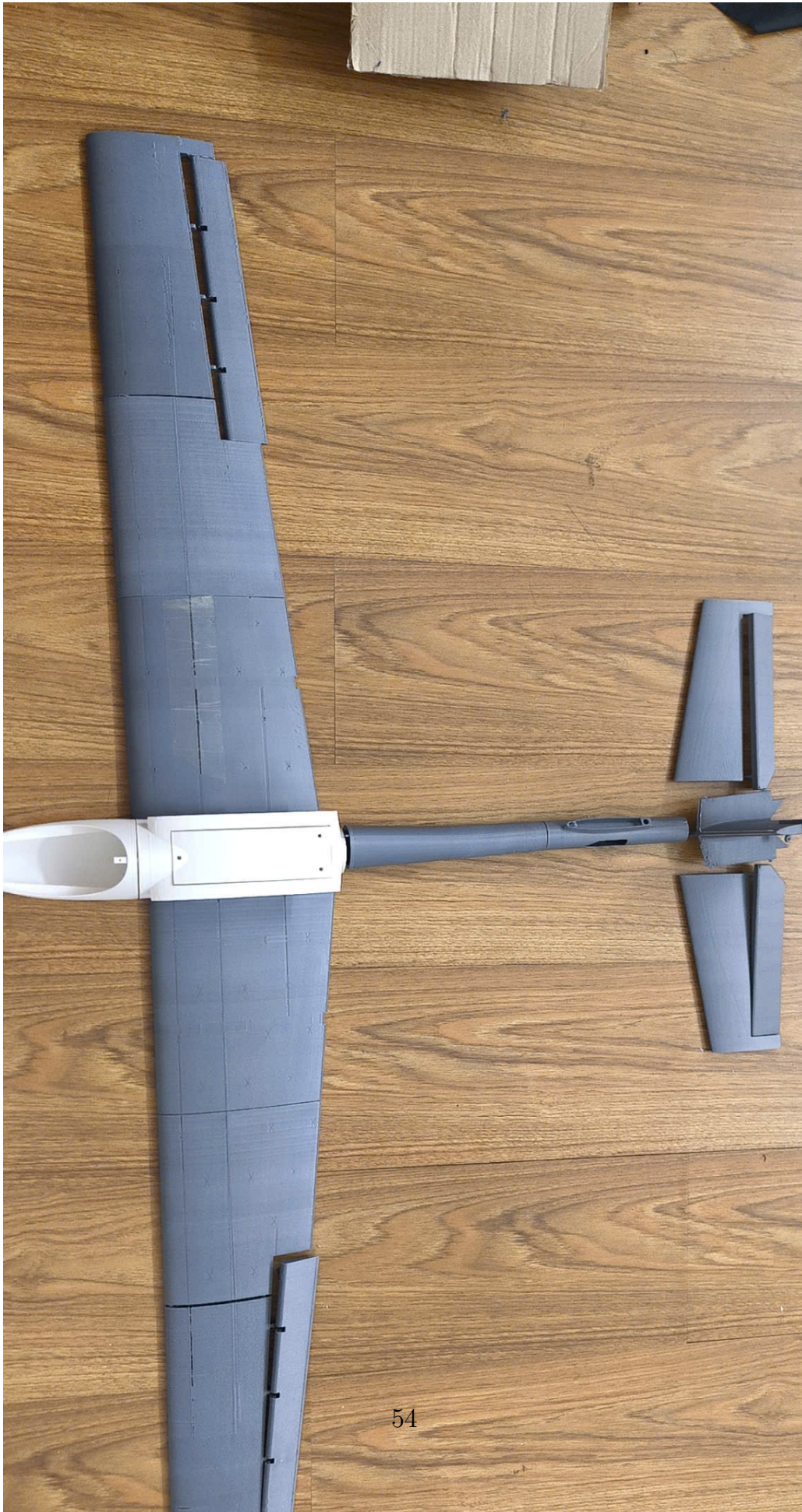
- [1] ArduPilot Dev Team. ArduPilot Plane and QuadPlane documentation. <https://ardupilot.org/plane/>, 2026. Accessed 2026.
- [2] ArduPilot Dev Team. ardupilot\_gazebo: Gazebo plugin for ArduPilot SITL. [https://github.com/ArduPilot/ardupilot\\_gazebo](https://github.com/ArduPilot/ardupilot_gazebo), 2026. Accessed 2026.
- [3] Open Robotics. Gazebo simulator documentation. <https://gazebo.org/>, 2026. Accessed 2026.
- [4] Randal W. Beard and Timothy W. McLain. *Small Unmanned Aircraft: Theory and Practice*. Princeton University Press, 2012.
- [5] Robert Mahony, Vijay Kumar, and Peter Corke. Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor. *IEEE Robotics & Automation Magazine*, 19(3):20–32, 2012. doi: 10.1109/MRA.2012.2206474.
- [6] Thomas Lombaerts, John Kaneshige, Stefan Schuet, Gordon Hardy, Bimal Aponso, and Kimberlee Shish. Nonlinear dynamic inversion based attitude

- control for a hovering quad tiltrotor eVTOL vehicle. In *AIAA Scitech 2019 Forum*, San Diego, CA, 2019. doi: 10.2514/6.2019-0134. AIAA 2019-0134.
- [7] Lei Zhou, Jitao Yang, Torben Strampe, et al. Incremental nonlinear dynamic inversion based path-following control for a hybrid quad-plane unmanned aerial vehicle. *International Journal of Robust and Nonlinear Control*, 33(17): 10304–10327, 2023. doi: 10.1002/rnc.6863.
- [8] A. A. Lambregts. Vertical flight path and speed control autopilot design using total energy principles. In *AIAA Guidance and Control Conference*, 1983. AIAA 1983-2239.
- [9] Agnes Steinert, Stefan Raab, Simon Hafner, Florian Holzapfel, and Haichao Hong. From fundamentals to applications of incremental nonlinear dynamic inversion: A survey on INDI – part I. *Chinese Journal of Aeronautics*, 38(11): 103553, 2025. doi: 10.1016/j.cja.2025.103553.
- [10] S. A. Snell, D. F. Enns, and William L. Garrard. Nonlinear inversion flight control for a supermaneuverable aircraft. *Journal of Guidance, Control, and Dynamics*, 15(4):976–984, 1992. doi: 10.2514/3.20932.
- [11] James M. Buffington. Modular control law design for the innovative control effectors (ICE) tailless fighter aircraft configuration 101-3. Technical Report AFRL-VA-WP-TR-1999-3057, Air Force Research Laboratory, Wright-Patterson AFB, 1999.
- [12] Patrick Smith and Alan Berry. Flight test experience of a nonlinear dynamic inversion control law on the VAAC Harrier. In *AIAA Guidance, Navigation, and Control Conference*, 2000. AIAA 2000-3914.
- [13] Patrick Smith. A simplified approach to nonlinear dynamic inversion based flight control. In *AIAA Guidance, Navigation, and Control Conference*, 1998. AIAA 1998-4461.
- [14] Barton Bacon and Aaron Ostroff. Reconfigurable flight control using nonlinear dynamic inversion with a special accelerometer implementation. In *AIAA Guidance, Navigation, and Control Conference*, 2000. AIAA 2000-4565.

- [15] S. Sieberling, Q. P. Chu, and J. A. Mulder. Robust flight control using incremental nonlinear dynamic inversion and angular acceleration prediction. *Journal of Guidance, Control, and Dynamics*, 33(6):1732–1742, 2010. doi: 10.2514/1.49978.
- [16] Agnes Steinert, Stefan Raab, Simon Hafner, Florian Holzapfel, and Haichao Hong. Advancements in incremental nonlinear dynamic inversion and its components: A survey on INDI – part II. *Chinese Journal of Aeronautics*, 38(11): 103591, 2025. doi: 10.1016/j.cja.2025.103591.
- [17] Ewoud J. J. Smeur, Qiping Chu, and Guido C. H. E. de Croon. Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles. *Journal of Guidance, Control, and Dynamics*, 39(3):450–461, 2015. doi: 10.2514/1.G001490.
- [18] Robin van ’t Veld, Erik-Jan Van Kampen, and Qi Ping Chu. Stability and robustness analysis and improvements for incremental nonlinear dynamic inversion control. In *AIAA Guidance, Navigation, and Control Conference*, 2018. AIAA 2018-1127.
- [19] Xuerui Wang, Erik-Jan van Kampen, Qi Ping Chu, and Ping Lu. Stability analysis for incremental nonlinear dynamic inversion control. *Journal of Guidance, Control, and Dynamics*, 42(5):1116–1129, 2019. doi: 10.2514/1.G003791.
- [20] OpenVSP Development Team. OpenVSP: Parametric aircraft geometry tool. <https://openvsp.org/>, 2026. Accessed 2026.



## A Physical Prototype Build



## B Vehicle and Controller Parameters

The PID runs use `sim/stock_pid.param`. The INDI runs use the same file with `sim/indi.param` appended, so the two differ only in the lines of Table 9. With `Q_INDI_ENABLE = 0` (the default) the INDI code is not loaded and the aircraft runs stock ArduPilot.

**Table 9.** INDI parameters, as flown (`sim/indi.param`).

| Parameter                              | Value   | Source     | Role                                                 |
|----------------------------------------|---------|------------|------------------------------------------------------|
| <code>Q_INDI_ENABLE</code>             | 1       | —          | Load the INDI controller at boot                     |
| <code>Q_INDI_AXES</code>               | 27      | —          | Bitmask: VTOL roll, VTOL pitch, FW roll              |
| <code>Q_INDI_FILT_HZ</code>            | 12.7 Hz | design     | Two poles of $H$ (Eq. (14))                          |
| <code>Q_INDI_RLL_G</code>              | 56.7    | identified | Lift-motor roll effectiveness                        |
| <code>Q_INDI_PIT_G</code>              | 44.8    | identified | Lift-motor pitch effectiveness                       |
| <code>Q_INDI_YAW_G</code>              | 0       | excluded   | Zero keeps yaw on the PID                            |
| <code>Q_INDI_ACT_TC</code>             | 0       | identified | Lift-motor lag                                       |
| <code>Q_INDI_ACT_DLY</code>            | 4 ms    | identified | Lift-motor delay                                     |
| <code>Q_INDI_RLL_K, PIT_K</code>       | 13.5/s  | margins    | Rate error to $\nu$                                  |
| <code>Q_INDI_FW_RLL_G</code>           | 16.7    | identified | Aileron effectiveness at <code>SCALING_SPEED</code>  |
| <code>Q_INDI_FW_PIT_G</code>           | 12.4    | identified | Elevator effectiveness at <code>SCALING_SPEED</code> |
| <code>Q_INDI_FW_TC</code>              | 10 ms   | identified | Surface servo lag                                    |
| <code>Q_INDI_FW_RLL_K, FW_PIT_K</code> | 10.5/s  | margins    | Rate error to $\nu$                                  |

Effectiveness is in  $\text{rad/s}^2$  per unit normalised command: full mixer input for the lift motors, full surface travel for the surfaces. `SCALING_SPEED` is 15 m/s; the surface values are divided by the square of ArduPilot’s speed scaler, which is Eq. (16). “Identified” values come from `analysis/identify_G.py` on the hover and fixed-wing system-identification runs; “margins” are the largest gains with at least 6 dB gain margin and  $45^\circ$  phase margin from `analysis/indi_margins.py`, reduced slightly.

**Table 10.** Other settings referred to in the text, as flown (`sim/stock_pid.param`).

| Parameter       | Value             | Plain name in the text                               |
|-----------------|-------------------|------------------------------------------------------|
| AIRSPEED_STALL  | 12 m/s            | stall speed                                          |
| AIRSPEED_MIN    | 14 m/s            | minimum airspeed                                     |
| AIRSPEED_CRUISE | 17 m/s            | cruise airspeed                                      |
| Q_ASSIST_SPEED  | 13 m/s            | lift-motor assist speed                              |
| Q_OPTIONS       | 0 (bit 7 cleared) | forced-assist setting (bit 7, Q_ASSIST_FORCE_ENABLE) |
| Q_TRANSITION_MS | 3000 ms           | timed part of the forward transition                 |
| SCHED_LOOP_RATE | 400 Hz            | nominal control-loop rate                            |
| INS_GYRO_FILTER | 20 Hz             | gyro low-pass filter ( $F_g$ )                       |
| WP_RADIUS       | 25 m              | waypoint acceptance radius                           |
| NAVL1_PERIOD    | 12 s              | navigation look-ahead period                         |

## C Names Used in ArduPilot, Gazebo and the Project Files

The main text uses plain names. This appendix gives the matching name in the code, the model file or the log. File paths are relative to the `rework/` folder.

| Plain name in the text               | Name in ArduPilot, Gazebo or the repository |
|--------------------------------------|---------------------------------------------|
| <i>Flight modes and log messages</i> |                                             |
| hover mode                           | QHOVER                                      |
| stabilised fixed-wing mode           | FBWA                                        |
| logged airspeed                      | CTUN.As                                     |
| airspeed-sensor messages             | ARSP                                        |
| stored hover throttle                | Q_M_THST_HOVER                              |
| batch-sampled gyro log               | INS_LOG_BAT_MASK                            |
| logged vibration levels              | VIBE.VibeX/Y/Z                              |
| logged body rates                    | RATE                                        |
| parameters written at boot           | PARM                                        |
| INDI log, lift motors / surfaces     | INDI / INDF                                 |

*Controller code (branch *indi*)*

|                                          |                                                                                         |
|------------------------------------------|-----------------------------------------------------------------------------------------|
| INDI core, one per axis                  | AC_INDI_Axis (AC_INDI.cpp)                                                              |
| INDI parameter block                     | AC_INDI_Params, parameters Q_INDI_*                                                     |
| lift-motor INDI rate loop                | AC_AttitudeControl_INDI::rate_controller_run_dt()<br>derived from AC_AttitudeControl_TS |
| surface INDI rate loop                   | AP_FW_Controller::run_rate_control()                                                    |
| boot-time switch                         | QuadPlane::setup()                      reading<br>Q_INDI_ENABLE                        |
| applied mixer input $u_{\text{applied}}$ | AC_AttitudeControl_INDI::applied_input()                                                |

---

*Gazebo model (sim/gazebo\_models/VTOL\_Quadplane/model.sdf)*

|                                                           |                                                                |
|-----------------------------------------------------------|----------------------------------------------------------------|
| lift-drag system                                          | gz-sim-lift-drag-system                                        |
| $S, \alpha_0, C_{L\alpha}, C_{D\alpha}, C_{m\alpha}$      | area, a0, cla, cda, cma                                        |
| $\alpha_{\text{stall}}$ , post-stall slopes               | alpha_stall,    cla_stall,    cda_stall,<br>cma_stall          |
| $\mathbf{r}_{cp}, \hat{\mathbf{e}}_f, \hat{\mathbf{e}}_u$ | cp, forward, upward                                            |
| $k_\delta$                                                | control_joint_rad_to_cl                                        |
| servo scaling                                             | ArduPilotPlugin                      <multiplier>,<br><offset> |
| Gazebo wind plugin                                        | WindEffects, links with enable_wind                            |

---

*Files and scripts*

|                                |                                                                     |
|--------------------------------|---------------------------------------------------------------------|
| square mission                 | sim/square.plan                                                     |
| zigzag mission                 | sim/Default square zigzag path.plan                                 |
| mission runner, pulse schedule | sim/fly_plan_mission.py                      (class<br>Disturbance) |
| torque injection               | sim/disturb.py                                                      |
| lifting-surface ranking        | analysis/auxiliary/rank_lifting_surfaces.py                         |
| mission geometry               | analysis/auxiliary/plan_geometry.py                                 |
| autopilot stored memory        | eprom.bin                                                           |

## D Transition Control Hooks

The effector-ownership rule of Listing 1 is evaluated in `Plane::stabilize()` (`ArduPlane/Attitude.cpp`) every loop:

```

1 fw_indi = quadplane.available() and Q_INDI_ENABLE > 0
2         and not quadplane.in_vtol_mode()
3         and quadplane.transition->complete()
4         and not quadplane.in_assisted_flight()
5 rollController.set_indi(params, ROLL, fw_indi and FW_ROLL
6   in Q_INDI_AXES)
pitchController.set_indi(params, PITCH, fw_indi and FW_PITCH
  in Q_INDI_AXES)

```

**Listing 2.** Surface INDI permission, as in `Plane::stabilize()`.

| Name                                   | Function                                                                                                                  |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>in_vtol_mode()</code>            | True in any hover flight mode (take-off, landing, hover).                                                                 |
| <code>transition-&gt;complete()</code> | True once the forward transition has finished.                                                                            |
| <code>in_assisted_flight()</code>      | True while the lift motors help the wing (below <code>Q_ASSIST_SPEED</code> , or forced by <code>Q_OPTIONS</code> bit 7). |
| <code>SLT_Transition</code>            | Transition state machine: wait for airspeed, timed handover ( <code>Q_TRANSITION_MS</code> ), done.                       |
| <code>AIRSPPEED_MIN</code>             | Airspeed at which the airspeed wait ends.                                                                                 |
| <code>set_indi()</code>                | Tells a surface rate controller whether INDI may run this cycle; otherwise the stock PID runs.                            |

The lift-motor loop needs no gate of its own: when the transition is complete and assist is off, ArduPilot stops running the VTOL rate loop, and `AC_AttitudeControl_INDI` additionally falls back to the PID whenever the motors are not fully spun up.

## E Gazebo, SITL, and MAVLink Configuration

This appendix will reproduce the relevant SDF blocks (lift-drag plugin parameters per surface, motor/propeller plugin parameters, sensor rates), the ArduPilot parameter file, the MAVLink/serial port configuration used to connect SITL to Gazebo, and the exact launch commands, once the corresponding files are finalised.

## F Additional Derivations and Pseudocode

Listing 3 gives the per-cycle update of one INDI axis (`AC_INDI_Axis::update()`), including the command synchronisation of Eq. (15), and the per-axis fallback of the lift-motor loop.

```
1 function indi_axis_update(rate, u_applied, nu, G, dt):
2     # dt is the measured loop period (3 ms in SITL, not the
      nominal 2.5 ms)
3     u_act    = actuator_model(u_applied)           # A: delay,
      then first-order lag
4     u_gyro   = gyro_filter_copy(u_act)             # F_g: same
      low-pass as the gyro
5     u0_f     = H(u_gyro)                           # H: two
      poles at FILT_HZ
6     wdot     = (rate - rate_prev) / dt             # backward
      difference
7     wdot_f   = H(wdot)                             # same H as
      the command path
8     rate_prev = rate
9     return u0_f + (nu - wdot_f) / G                # new
      normalised command
10
11 function lift_motor_rate_loop(rate_target, gyro, dt):      #
      every VTOL cycle
12     for axis in (roll, pitch, yaw):
13         u_applied = recovered_from_motor_outputs(axis)
14         if motors_spooled and axis in Q_INDI_AXES and G[axis]
            > 0:
15             nu = K[axis] * (rate_target[axis] - gyro[axis])
16             out[axis] = indi_axis_update(gyro[axis],
                u_applied, nu, G[axis], dt)
17             pid[axis].reset_integrator()             # bumpless
                hand-back
18         else:
19             out[axis] = pid[axis].update(rate_target[axis],
                gyro[axis])
20             indi[axis].reset(gyro[axis], u_applied)    #
```

```

21         bumpless switch-in
22         mixer.set(out)
23 function surface_rate_loop(axis, rate_target, rate,
24     speed_scaler, dt):
25     if fw_indi_allowed(axis) and speed_scaler > 0 and
26         FW_G[axis] > 0:
27         G = FW_G[axis] / speed_scaler^2           # Eq.
28             (fw-indi-scaling)
29         nu = FW_K[axis] * (rate_target - rate)
30         return indi_axis_update(rate, u_last, nu, G, dt)
31     return stock_pid(axis)                         # INDI
32     states follow the PID

```

**Listing 3.** One INDI axis, one call per control cycle.